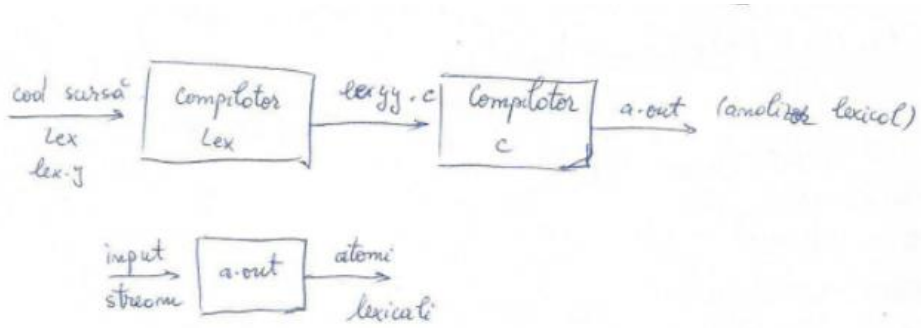
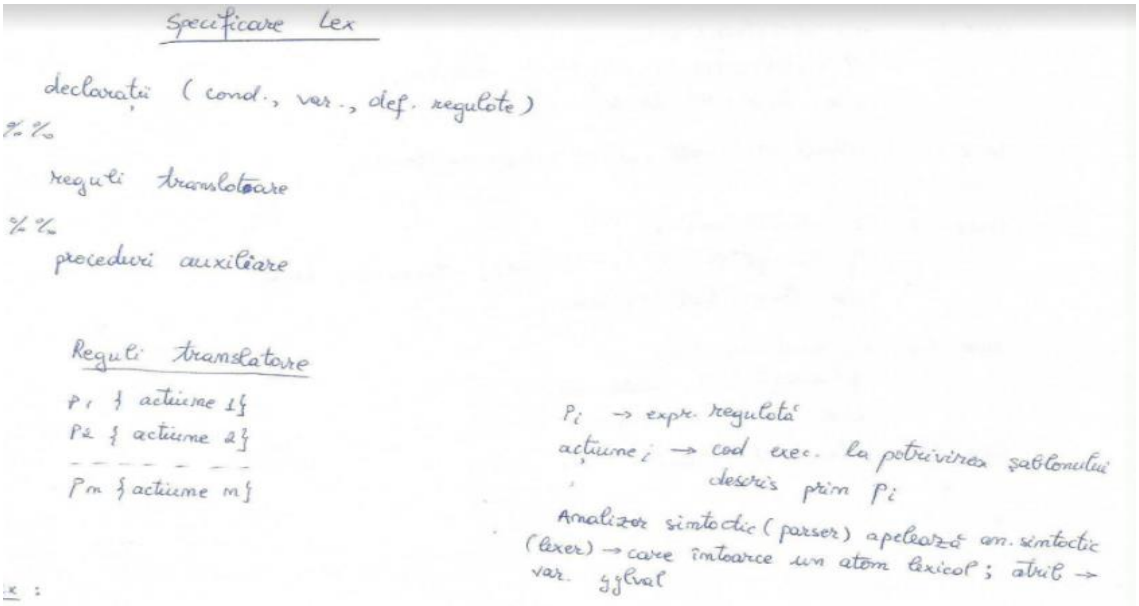
Analiza lexicala – lex

(cod sursa, compilator lex, compilator c. analizator lexical)



Specificare Lex (reguli translatare)



Def constante DELIM, WS, LIT, CIF, ID, NUM, WS, THEN, ELSE, <, >, LONG INSTAL_ID

```
%{ * def. constante LT, LE, EQ, NE, GT, GE, IF, THEN, ELSE, ID, NUM */
%}

delim   [ \t \n]
ws      {delim}+
lit     [A-Z a-z]
cif     [0-9]
id      {lit}({lit}|{cif})*
num     {cif}+ (\.{cif}+)?(E[+\-]?{cif}+)?

%%
{ws}    {/* nu executa nimic */}
if      {return IF;}
then    {return THEN;}
else    {return ELSE;}
{id}    {yylval = instal_id(); return ID;}
{num}   {yylval = instal_num(); return NUM;}
"<"     {yylval = LT; return OPREL;}
"<="    {yylval = LE; return OPREL;}
">"     {yylval = GT; return OPREL;}
```

+ = o dată, cel puțin

? = care poate să lipsească


```
%%
long instal_id() {/* pune în TS lexema cuprinsă între yytext și yytext+
                    yyleng */}
long instal_num() {/* pune lexema asoc. unui nr. în tabelă */}
```

obs: trebuie să prefixăm punctul (\.) pt. a avea semnificația de pct, altfel = înseamnă toate caracterele cu excepția NL (new line)

\- (minus), altfel → interval

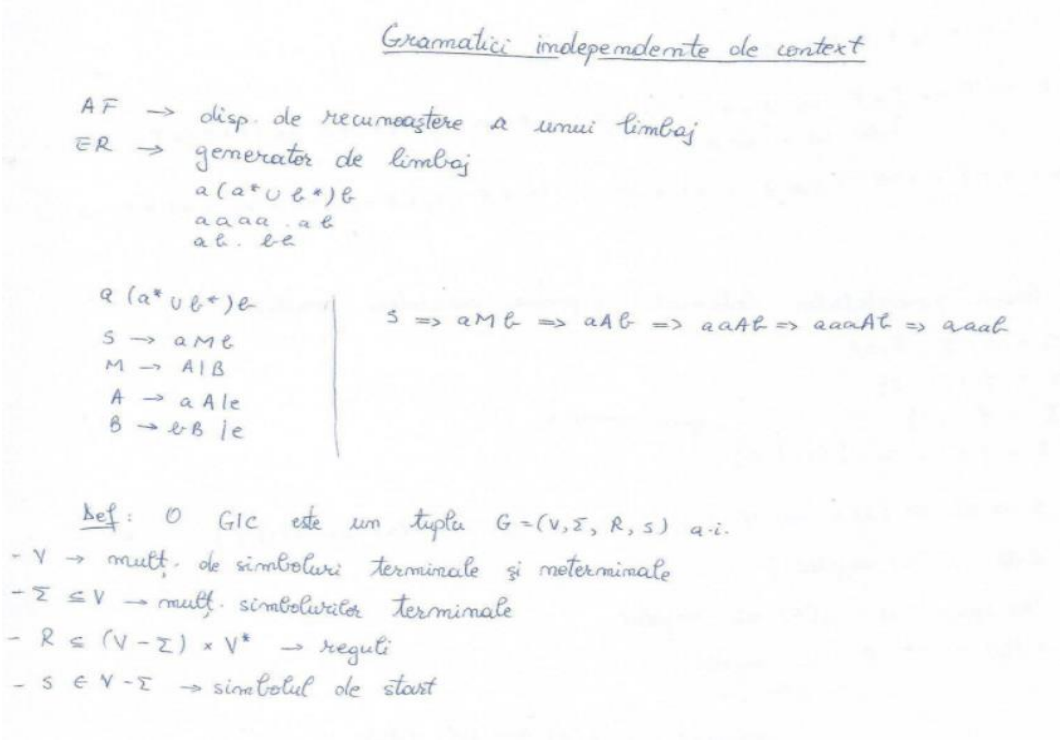
Reguli translatare: {} → definiția
șir de caractere (" eventual") → potrivire

yylval, yytext, yyleng } tip long

Gramatici independente de context

AF, ER, generator de limbaj

GIC este un tuplu (multimer de simboluri terminale si neterminale)



Gramatici independente de context

AF → disp. de recunoaștere a unui limbaj

ER → generator de limbaj

$a(a^* \cup b^*)b$

aaaa . ab

ab . bb

$a(a^* \cup b^*)b$

$S \to aMb$

$M \to A \mid B$

$A \to aA \mid \varepsilon$

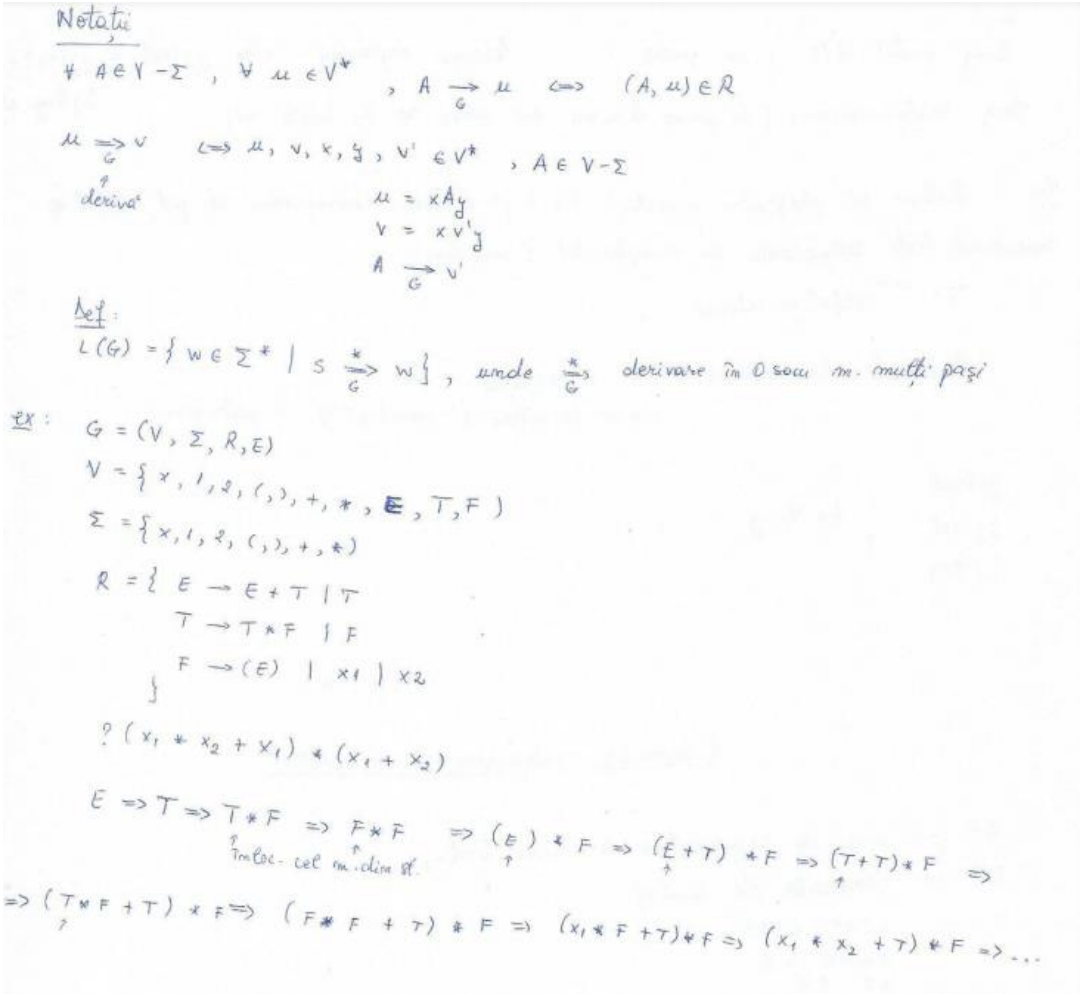
$B \to bB \mid \varepsilon$

$S \Rightarrow aMb \Rightarrow aAb \Rightarrow aaAb \Rightarrow aaaAb \Rightarrow aaab$

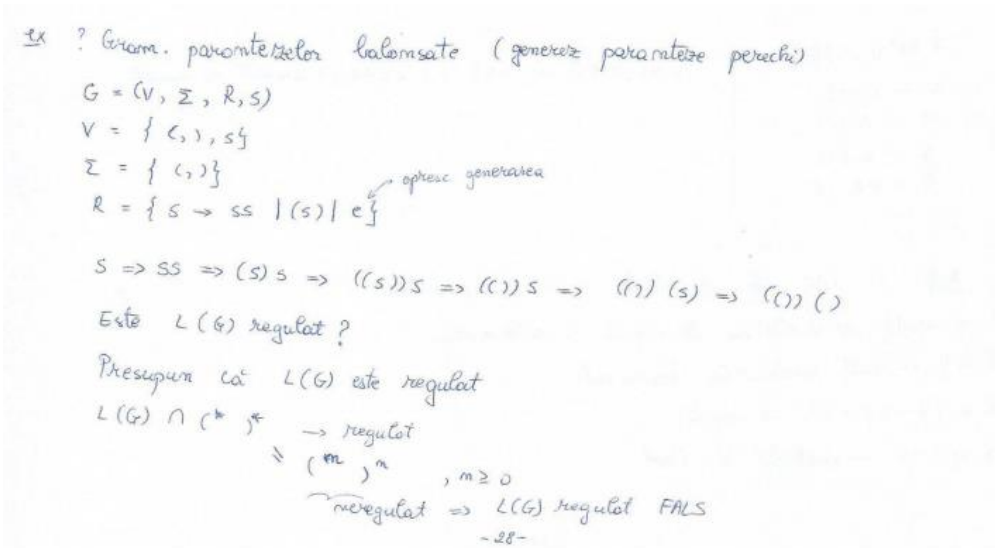
Def: O GIC este un tuplu $G = (V, \Sigma, R, S)$ a.i.

- V → mult. de simboluri terminale și neterminale
- $\Sigma \subseteq V$ → mult. simbolurilor terminale
- $R \subseteq (V - \Sigma) \times V^*$ → reguli
- $S \in V - \Sigma$ → simbolul de start

Notatii, ex G = (v, r, e)

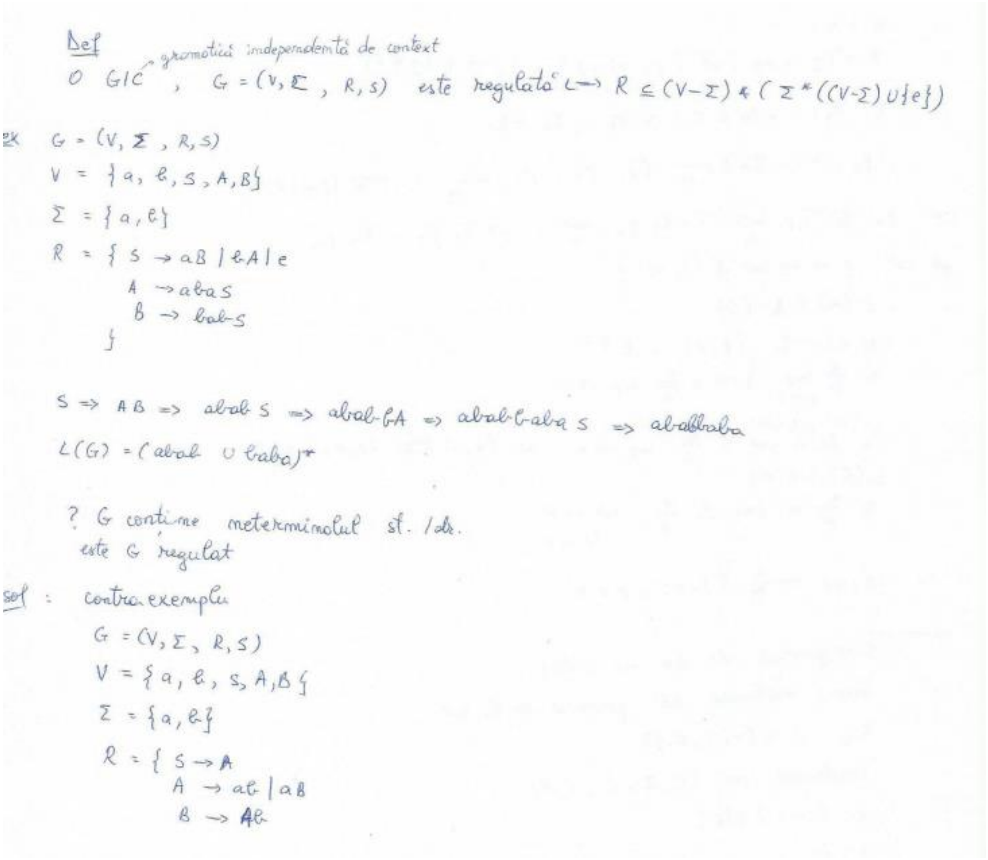


Gram paranteze lansate (paranteze perechi)



Gramatici regulate

O gic este regulata (exemplu, contraexemplu)



Def
O GIC (gramatică independentă de context), $G = (V, \Sigma, R, S)$ este regulată $\Leftrightarrow R \subseteq (V-\Sigma) \times (\Sigma^* ((V-\Sigma) \cup \{\varepsilon\}))$

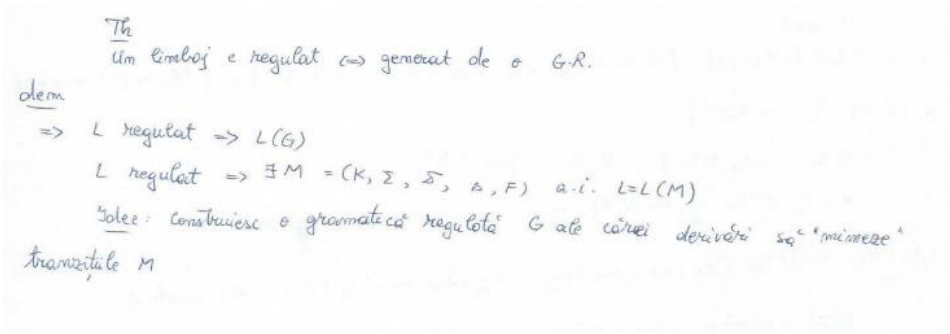
ex $G = (V, \Sigma, R, S)$
$V = \{a, b, S, A, B\}$
$\Sigma = \{a, b\}$
$R = \{ S \to aB \mid bA \mid \varepsilon$
$A \to abaS$
$B \to babS$
$\}$

$S \Rightarrow AB \Rightarrow ababS \Rightarrow ababbA \Rightarrow ababbaba S \Rightarrow ababbaba$
$L(G) = (abab \cup baba)^*$

? G conține neterminalul st. / dr.
este G regulat

sol : contra-exemplu
$G = (V, \Sigma, R, S)$
$V = \{a, b, S, A, B\}$
$\Sigma = \{a, b\}$
$R = \{ S \to A$
$A \to ab \mid aB$
$B \to Ab$

Teorema : un limbaj este regulat ⇔ generat de o G.R(dem)



Th
Un limbaj e regulat $\Leftrightarrow$ generat de o G.R.

dem
$\Rightarrow$ L regulat $\Rightarrow L(G)$
L regulat $\Rightarrow \exists M = (K, \Sigma, \delta, s, F)$ a.î. $L = L(M)$
Idee: Construiesc o gramatică regulată G ale cărei derivări să "mimeze" tranzițiile M

$G = (V, \Sigma, R, S)$

$V = K \cup \Sigma$

$S = s$

$R = \{q \to ap \mid \delta(q, a) = p\} \cup \{q \to e \mid q \in F\}$

$\forall p_0, \dots, p_m \in K, \forall \sigma_1, \dots, \sigma_m \in \Sigma$

$(p_0, \sigma_1 \dots \sigma_m) \vdash_M (p_1, \sigma_2 \dots \sigma_m) \vdash_M \dots \vdash_M (p_m, e)$

$\Leftrightarrow p_0 \Rightarrow_G \sigma_1 p_1 \Rightarrow_G \sigma_1 \sigma_2 p_2 \Rightarrow_G \dots \Rightarrow_G \sigma_1 \sigma_2 \dots \sigma_m p_m$

pt. că $p \to aq \Leftrightarrow \delta(p, a) = q$

$L(M) \subseteq L(G)$

$(s, w) \vdash_M^* (p, e)$, $p \in F$

$s \Rightarrow_G^* wp \mid \Rightarrow s \Rightarrow_G^* wp \Leftrightarrow w$
$p \to e$

$L(G) \subseteq L(M)$

$s \Rightarrow_G^* w \Leftrightarrow s \Rightarrow_G^* wp \Rightarrow w \Rightarrow (s, w) \vdash_M^* (p, e)$, $p \in F$
$\uparrow p \to e$

$L(G) \subseteq L(M)$

$s \Rightarrow_G^* w \Leftrightarrow s \Rightarrow_G^* wp \Rightarrow w$
$\uparrow p \to e$

$\Rightarrow (s, w) \vdash_M^* (p, e)$, $p \in F$

L → generat de $G_R \Rightarrow L(M)$

Idee: construiesc AF pornind de la G_R

Fie $G = (V, \Sigma, R, S)$

Construiesc $M = (K, \Sigma, \Delta, s, F)$

$K = (V - \Sigma) \cup \{f\}$

$s = S$

$F = \{f\}$

$\Delta = \{(A, w, B) \mid A \to wB \in R, A, B \in V - \Sigma, w \in \Sigma^*\} \cup \{(A, w, f) \mid A \to w \in R, A, B \in V - \Sigma, w \in \Sigma^*\}$

$\forall A_1, \dots, A_m \in V - \Sigma, \forall w_1, \dots, w_m \in \Sigma^*$

$A_1 \Rightarrow_G w_1 A_2 \Rightarrow_G w_1 w_2 A_3 \Rightarrow_G \dots \Rightarrow_G w_1 \dots w_{m-1} A_m \Rightarrow_G w_1 \dots w_m \Leftrightarrow$

$(A_1, w_1 \dots w_m) \vdash_M (A_2, w_2 \dots w_m) \vdash_M \dots \vdash_M (A_m, w_m) \vdash_M (f, e)$ doar cuvinte Δ

$L(G) \subseteq L(M)$ și $L(M) \subseteq L(G)$

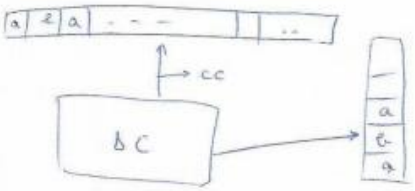
$S \Rightarrow^* w \Leftrightarrow (s, w) \vdash_M^* (f, e)$ și $(s, w) \vdash_M^* (f, e) \Leftrightarrow S \Rightarrow_G^* w$

$L(AFD) = L(AFN) = L(ER) = L(GR)$

Automate cu stiva (push-down)
Def: un Apd este un tuplu M = (k, …)
Multimea, alfabetul, starilor finale, rel de tranzitie, submultim finite

Automate cu Stivă (push-down)

$L = \{w \in \Sigma^* \mid w = w^R\}$



Def

Un APD este un tuplu $M = (K, \Sigma, \Gamma, \Delta, s, F)$

K → mulțimea finită a stărilor
Σ → alfabetul de intrare
Γ → alfabetul stivei
$s \in K$ → st. inițială
$F \in K$ → mult. stărilor finale
$\Delta \subseteq ((K \times \Sigma^* \times \Gamma^*) \times (K \times \Gamma^*))$ → rel. de tranziție
(submult. finită)

$((p, \beta), (q, \delta, \gamma))$

$((p, u, e), (q, a))$ → pune a pe stivă
$((p, u, a), (q, e))$ → scoate a de pe stivă

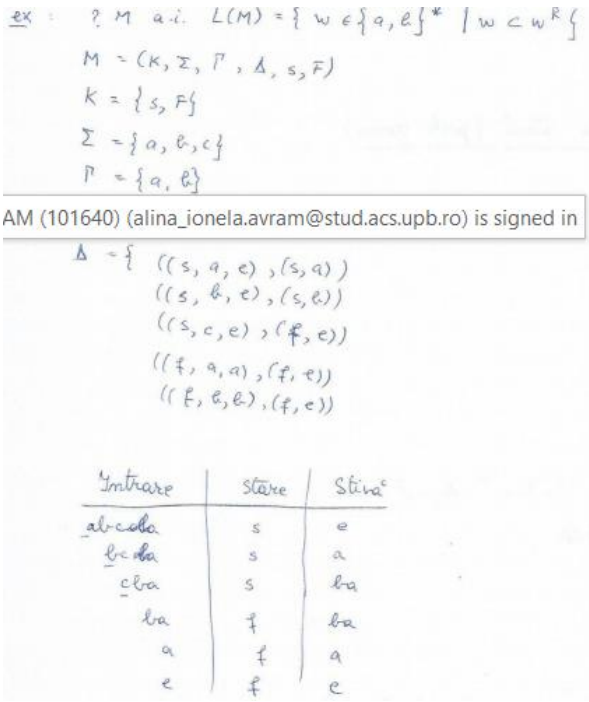
Config unui apd este un element din
Un sir w este acceptat de un apd
Un AF este un Apd care nu lucreaza cu stiva

Def
Config. unui APD este un element din $K \times \Sigma^* \times \Gamma^*$

Def
Un șir w este acceptat de un APD, M, $(s, w, e) \vdash_M^* (f, e, e)$, $f \in F$

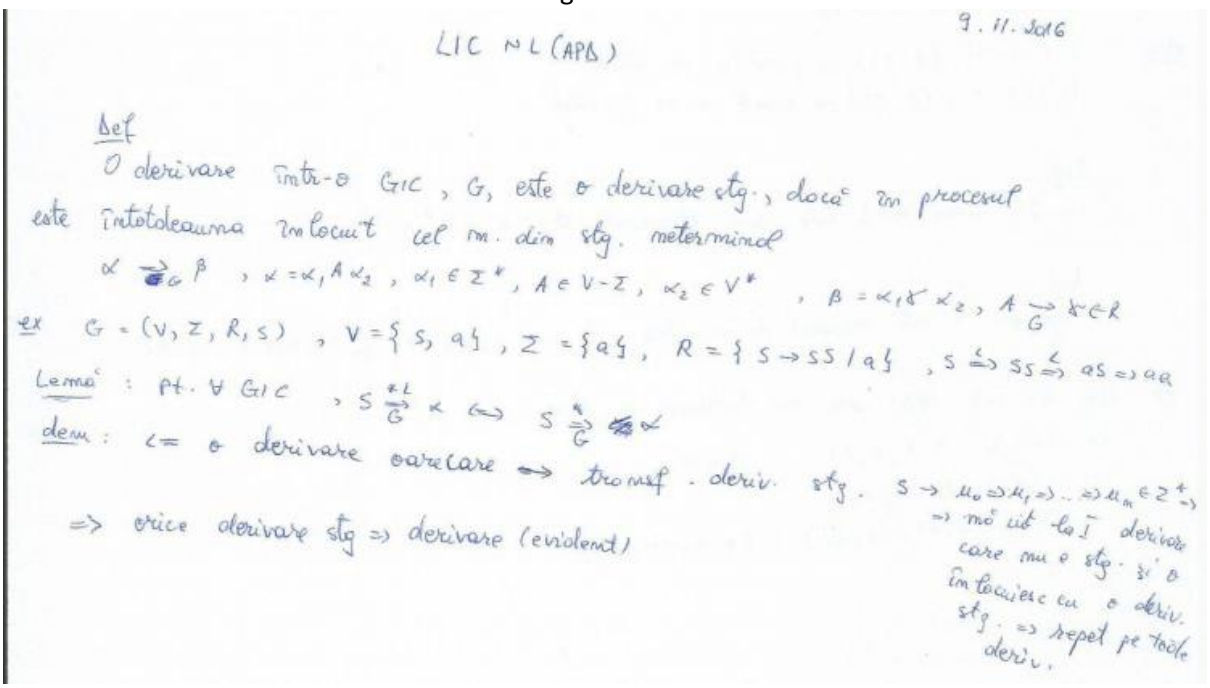
ex: Un AF este APD care nu lucrează cu stiva
$M = (K, \Sigma, \Gamma, \Delta, s, F)$, $\Gamma = \emptyset$
$M' = (K, \Sigma, \Delta', s, F)$
$\Delta = \{((p, u, e), (q, e)) \mid (p, u, q) \in \Delta'\}$

M astfel incat L(M) = {w apartine …w inclus ..} intrare stare stiva



Lic ~ L(APD)

Def: o derivare intr-o GIC …G este o derivare stg



Th: Clasa LIC este clasa L(APD)

Demonstratie, lema: oricare lic este acceptat de un apd

Th
Clasa LIC este clasa L(APΔ)

demo :

Lemă
∀ LIC este acceptat de un APΔ

demo :

Fie $G = (V, \Sigma, R, S)$ o GIC

Vreau să constr. $M = (K, \Sigma, \Gamma, \Delta, s, F)$ a.i. $L(M) = L(G)$

M are 2 stări p, q

$M = (\{p,q\}, \Sigma, V, \Delta, p, \{q\})$

$\Delta =$ 1° $((p, e, e)(q, S))$

2° $((q, e, A), (q, x))$, $A \xrightarrow{G} x \in R$

3° $((q, a, a)(q, e))$, $a \in \Sigma$

ex : $G = (V, \Sigma, R, S)$

$V = \{S, a, b, c\}$

$\Sigma = \{a, b, c\}$

$R = \{S \to aSa \mid bSb \mid c\}$, $L(G) = \{wcw^R \mid w \in \{a,b\}^*\}$

? M a.i. $L(M) = L(G)$

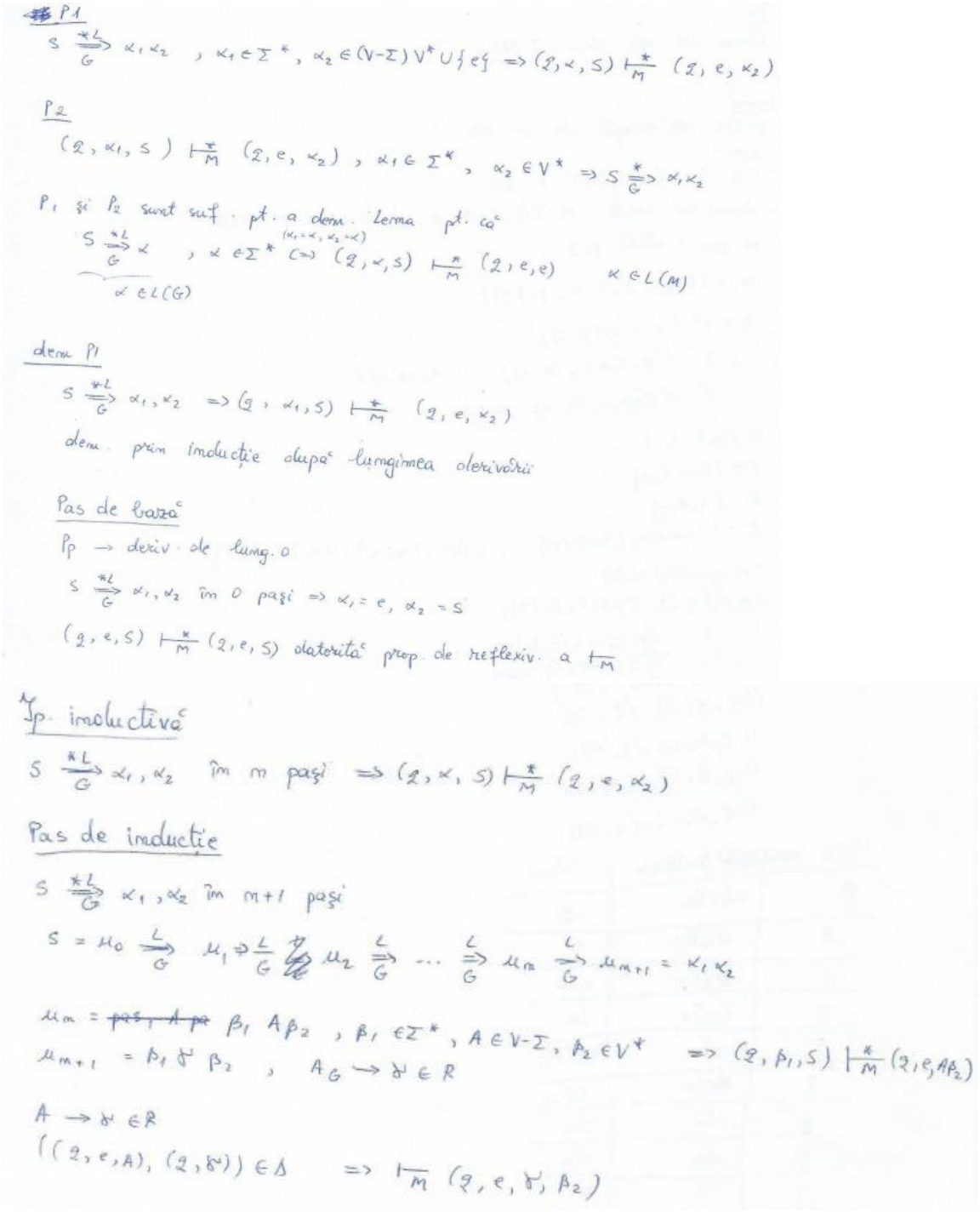
$M = (\{p, q\}, \Sigma, V, \Delta, p, \{q\})$

Δ : $((p, e, e), (q, S))$
$((q, e, S), (q, aSa))$
$((q, e, S), (q, bSb))$
$((q, e, S), (q, c))$
$((q, a, a), (q, e))$
$((q, b, b), (q, e))$
$((q, c, c), (q, e))$

Stare ~~intrare~~	Intrare	Stivă
p	abcba	e
q	abcba	S
q	abcba	aSa
q	bcba	Sa
q	bcba	bSba
q	~~b~~cba	Sba
q	cba	cba
q	ba	ba
q	a	a
q	e	e

P1 si P2 pentru a demonstra lema

Dem prin inductie dupa lungimea derivarii



$\beta_1 \gamma \beta_2 = \alpha_1 \alpha_2$

$|\alpha_1| \geq |\beta_1|$

$|\alpha_2| \leq |\gamma \beta_2|$

$\alpha_1 = \beta_1 \delta$

$\delta \alpha_2 = \gamma \beta_2$

$(q, \alpha_1, s) = (q, \beta_1 \delta, s) \vdash_M^* (q, \delta, A\beta_2) \vdash_M (q, \delta, \gamma\beta_2) \vdash_M^* (q, \delta, \delta\alpha_2) \vdash_M^* (q, e, \alpha_2)$

bim P_2 → temă

Lema: pentru orice limbaj acceptat de un apd pot construe o gic care sa-l genereze
Automat push-down stiva
Tranzitii intermediare / stari intermediare

Lemă

Pt. orice limbaj acceptat de un APD, pot construi o GIC care să-l genereze.

dem: $M = (K, \Sigma, \Gamma, \Delta, s, F)$ → APD simplu

1°. $((q, u, \beta), (p, \gamma))$, $|\beta| \leq 1$

2°. $((q, u, e), (p, \gamma)) \Rightarrow ((q, u, A), (q, \gamma A))$ | nu lucrează cu stiva ≡ pop, push

APD => transf. APDs (automat pop/push stivă)

$M = (K, \Sigma, P, \Delta, s, F)$

1° transf. $((q, u, \beta), (p, \gamma))$ pt. $|\beta| > 1$ transf. $\beta = B_1 \cdot B_m$

adaug tranziții interm. și stări intermediare

$(q, e, B_1), (t_1, e))$

$((t_1, e, B_2), (t_2, e))$

$((t_{m-1}, u, B_m), (p, \gamma)$ | → $e \in \Delta$, $t_1, \dots, t_{m-1} \in K$

pt. toate tranzițiile $|\beta| > 1$

2° $((q, u, e), (\beta, \gamma))$, înloc. $((q, u, A), (\beta, \gamma A))$

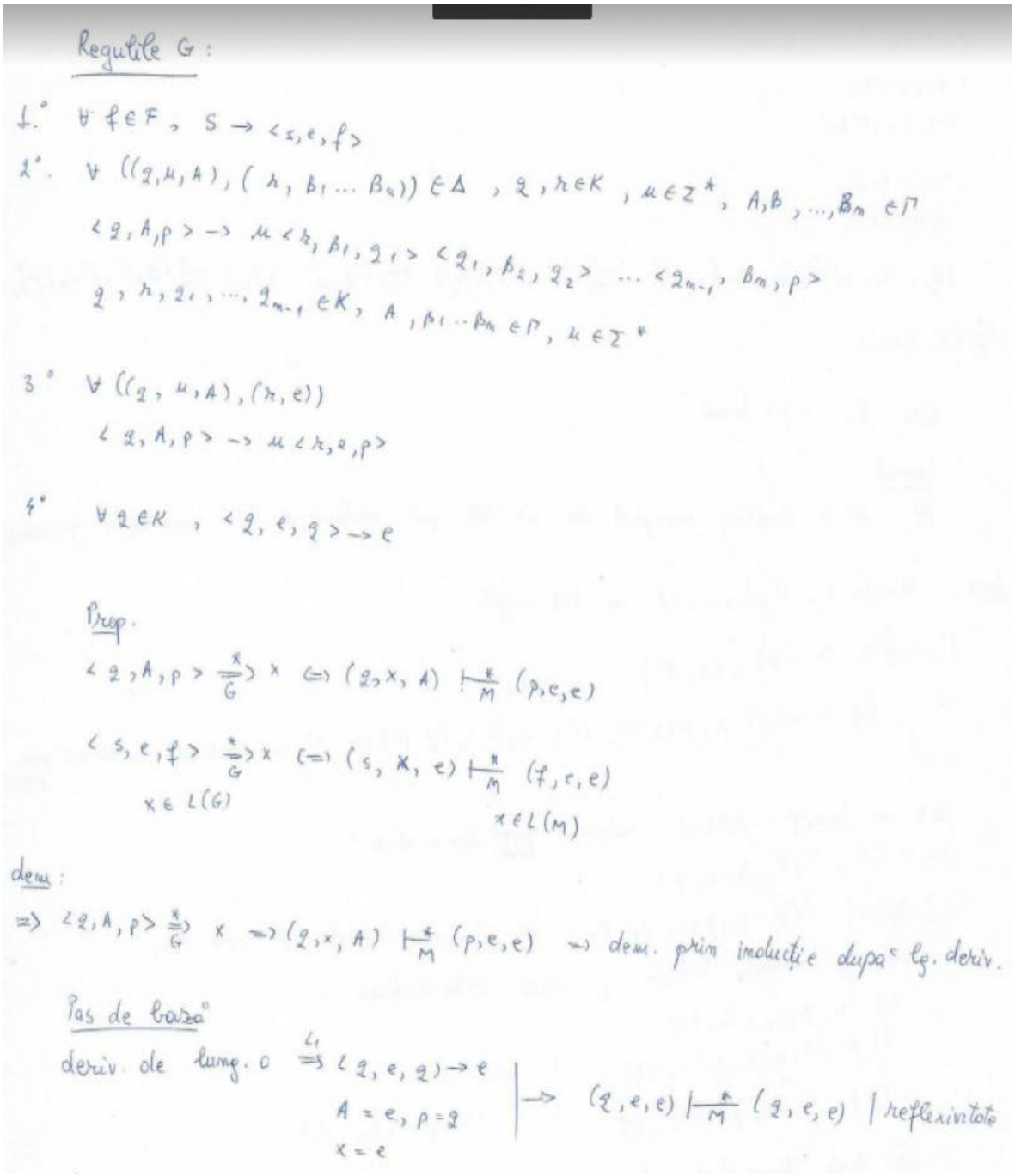
Fie M → APDs. Constr. G a.i. $L(M) = L(G)$

$G = (V, \Sigma, R, S)$

G conține pe lângă S și $\forall \in \Sigma$ neterm. $\langle q, A, p \rangle$ → șirul care e prelucrat de M între mom. de timp în care se află în starea q cu A pe stivă și ajunge în P, scoțând A de pe stivă.

$\langle q, e, p \rangle$ → gen. șirul care este prelucrat de M între mom. de timp în care se află în stările q și p, lăsând stiva nemodif.

Regulile G, pas de baza, ipoteza inductive, pas de inductie



Regulile G:

1° $\forall f \in F,\ S \to \langle s, e, f \rangle$

2° $\forall ((q, u, A), (r, B_1 \dots B_n)) \in \Delta,\ q, r \in K,\ u \in \Sigma^*,\ A, B, \dots, B_n \in \Gamma$

$\langle q, A, p \rangle \to u \langle r, B_1, q_1 \rangle \langle q_1, B_2, q_2 \rangle \dots \langle q_{n-1}, B_n, p \rangle$

$q, r, q_1, \dots, q_{n-1} \in K,\ A, B_1 \dots B_n \in \Gamma,\ u \in \Sigma^*$

3° $\forall ((q, u, A), (r, e))$

$\langle q, A, p \rangle \to u \langle r, e, p \rangle$

4° $\forall q \in K,\ \langle q, e, q \rangle \to e$

Prop.

$\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x \iff (q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$

$\langle s, e, f \rangle \overset{*}{\underset{G}{\Rightarrow}} x \iff (s, x, e) \overset{*}{\underset{M}{\vdash}} (f, e, e)$

$x \in L(G)$ $\qquad$ $x \in L(M)$

dem:

$\Rightarrow$ $\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x \to (q, x, A) \overset{*}{\underset{M}{\vdash}} (p, e, e)$ → dem. prin inductie dupa lg. deriv.

Pas de baza°

deriv. de lung. 0 $\overset{L_1}{\Rightarrow} \langle q, e, q \rangle \to e$, $A = e$, $p = q$, $x = e$ $\to (q, e, e) \overset{*}{\underset{M}{\vdash}} (q, e, e)$ | reflexivitate

Pas de inductie

$\langle q, A, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x$ în $k+1$ pași

Primul pas $< \begin{matrix} R_2 \\ R_3 \end{matrix}$

R2

$\langle q, A, p \rangle \overset{R_2}{\Rightarrow} u \, t \, \langle r, B_1, q_1 \rangle \langle q_1, B_2, q_2 \rangle \dots \langle q_{m-1}, B_m, p \rangle \overset{*}{\underset{G}{\Rightarrow}} x$

$\langle q_{i-1}, B_i, q_i \rangle \overset{*}{\underset{G}{\Rightarrow}} z_i$ în cel mult k pași $\overset{ip.ind.}{\Rightarrow}$

$(q_{i-1}, z_i, B_i) \vdash_M^* (q_i, e, e)$

$(q, u z \dots z_m, A) \vdash_M^* (r, z \dots z_m, B \dots B_m) \vdash_M^* (q_1, z_2 \dots z_m, b_2 \dots b_m) \vdash_M^*$

$\vdash_M^* (q_{m-1}, z_m, B_m) \vdash_M^* (p, e, e)$

R3

$\langle q, A, p \rangle \overset{R_3}{\underset{G}{\Rightarrow}} u \langle r, e, p \rangle \overset{+}{\underset{G}{\Rightarrow}} x$

Ψ z în cel mult k pași $\overset{ip.ind.}{\Rightarrow}$

$(q, u z, A) \vdash_M^* (r, z, e) \vdash_M^* (p, e, e)$

Proprietati de inchidere ale LIC

Reuniune/concatenare/ Kleene

Prop. de închidere ale LIC

Th

LIC sunt închise în rap. cu operațiile de:

a) reuniune
b) concatenare
c) kleene star

dem: a) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$

$(V_1 - \Sigma_1) \cap (V_2 - \Sigma_2) = \emptyset$

$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 \mid S_2\}, S)$

$S \overset{*}{\underset{G}{\Rightarrow}} w$, $w \in (\Sigma_1 \cup \Sigma_2)^* \iff S_1 \overset{*}{\underset{G}{\Rightarrow}} w$ sau $S_2 \overset{*}{\underset{G_2}{\Rightarrow}} w$

b) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G_2 = (V_2, \Sigma_2, R_2, S_2)$

$G = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S \to S_1 S_2\}, S)$

c) $G_1 = (V_1, \Sigma_1, R_1, S_1)$

$G = (V_1, \Sigma_1, R_1 \cup \{S_1 \to S_1 S_1 \mid e\}, S_1)$

Intersectia dintre LIC si LR este LIC

Th

Intersecția dintre LIC și LR este LIC.

dem: Fie $M_1 = (K_1, \Sigma, \Pi, \Delta_1, s_1, F_1)$

$M_2 = (K_2, \Sigma, \delta, s_2, F_2)$

Constr. M a.i. $L(M) = L(M_1) \cap L(M_2)$

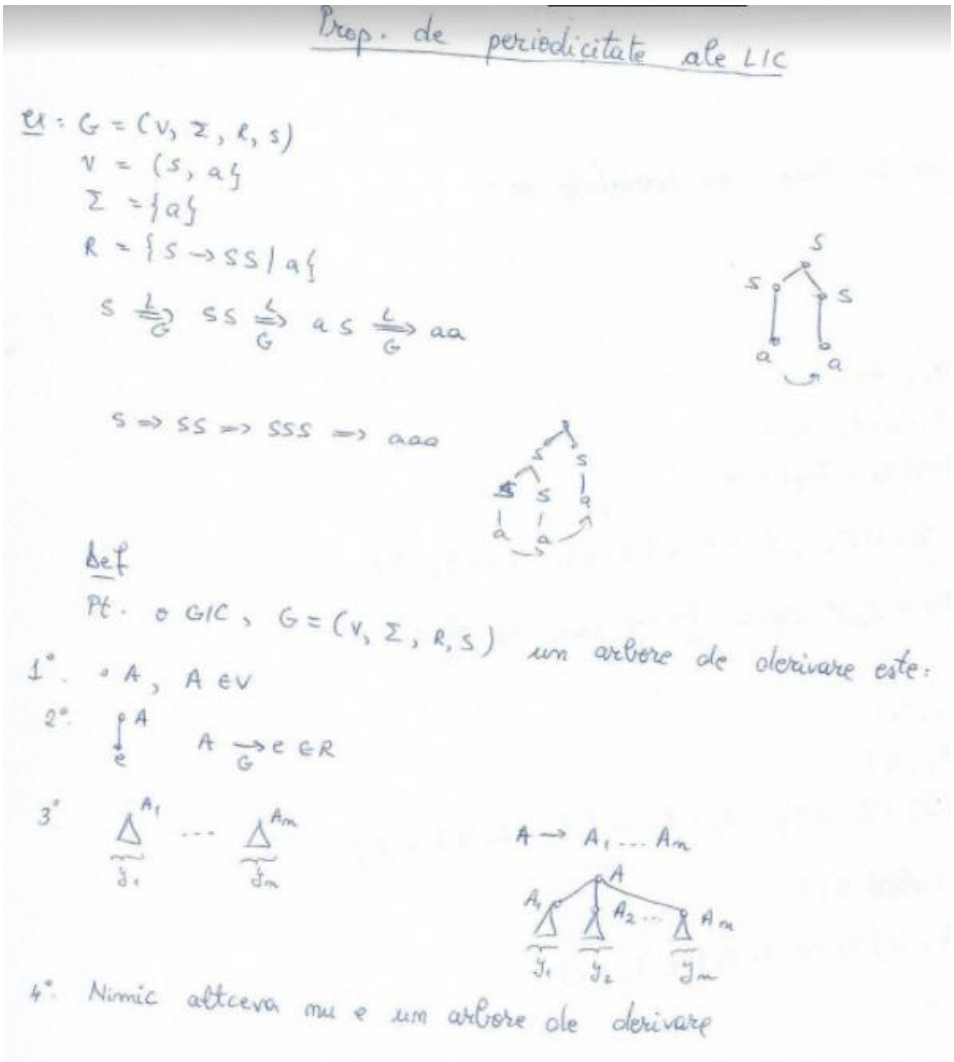
$M = (K_1 \times K_2, \Sigma, \Pi, \Delta, (s_1, s_2), F_1 \times F_2)$

$\Delta = (((q_1, q_2), u, \gamma), ((p_1, p_2), \beta)$

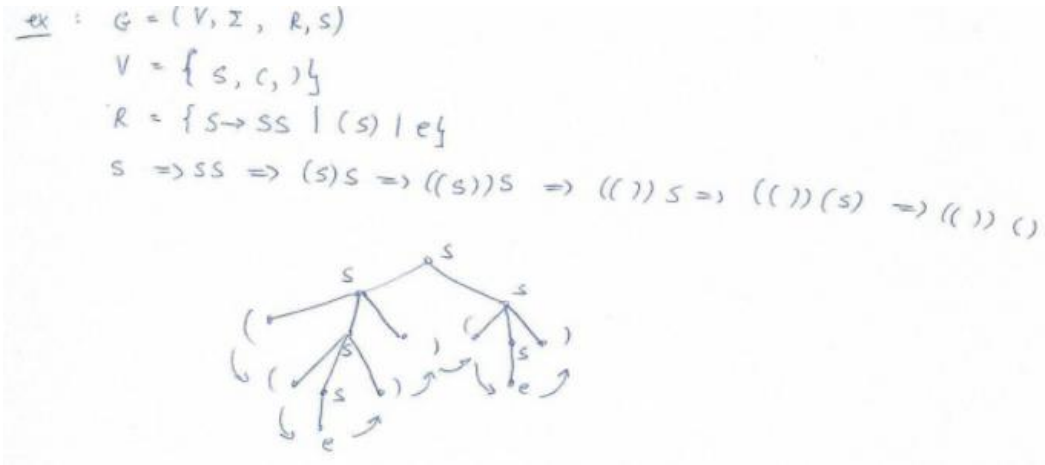
$((q_1, u, \gamma), (p_1, \beta)) \in \Delta_1$

$(q_2, u) \overset{*}{\underset{M}{\vdash}} (p_2, e)$

Proprietatea de peridiocitate ale LIC
Un arbore de derivare este



Ex ss



Lema de pompare
Constanta K dependenta
Un arbore de inaltime m are cel mult frunze
Se reduce

Th (Lema de pompare)

Fie $G \to$ GIC. At. $\exists$ o constanta K dependenta de G a.i. $\forall w \in L(G)$, $|w| > K$, $w = uvxyz$, $v \neq e$ sau $y \neq e$ și $uv^n xy^n \in L(G)$, $\forall n \geq 0$

dem

Fie $G = (V, \Sigma, R, S)$, $\exists K(G)$ a.i. $\forall w \in L(G)$ poate fi obț. printr-o derivare

$S \xrightarrow{*} uAz \xrightarrow{*} uvAyz \xrightarrow{*} w$

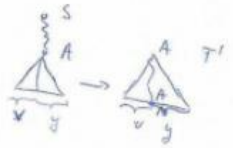
și $A \Rightarrow vAy$ poate fi repetata de ori cate ori

Notăm $p = \max\{|\alpha| \mid A \to \alpha \in R\}$

Un arb. de $\geq$ înălțime m are cel mult p^m frunze

$m = |V - \Sigma|$, $K = p^m$

Fie $w \in L(G)$, $|w| > K$



Nu se poate ca $v = e$ și $y = e$ pt. O alegere a căii.

$\Rightarrow$ arb. T' se reduce la o sg. cale de $\geq$ neterminale $\Rightarrow$ imposibil $|w| > K$

Limbajul independent context

Teorema

Limbajul $\{a^n b^n c^n \mid n \geq 0\}$ nu este ind. de context.

dem: Pres. $L \to$ LIC $\xrightarrow{\text{L. pompare}}$ $\exists K(G)$ a.i. $w \in L$, $|w| \geq K$, $w = uvxyz$, $v \neq e$ sau $y \neq e$ și $uv^i xy^i z \in L$, $\forall i \geq 0$

Fie $K \to$ const. (def. cf. LP) $\Rightarrow n > K/3$

$w = a^n b^n c^n$

v, y format ab sau bc $\to$ nu se poate $\to$ se strică forma șirurilor

v, y a/b/c $\to$ nu se poate, pt. că $u_a = u_b = u_c$, $\forall i \geq c$

$\Rightarrow L \to$ NU este LIC

Teorema

Limb. Ind. de Context nu sunt închise în rap. cu intersecția și complementarea.

dem: $L_1 = \{a^n b^n c^m \mid n, m > 0\}$

$L_2 = \{a^m b^n c^n \mid m, n > 0\}$

$L_1 \cap L_2 = \{a^n b^n c^n \mid n > 0\} \to$ ~~LIC~~

Complementare / proprietati algoritmice ale lic
Algoritmi care pot raspunde la urmatoarele intrebari l(gic)

Complementare

$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$

(închis) (închis)

Proprietăți algoritmice ale LIC

Teoremă

Există alg. care pot răspunde la urm. întrebări referitoare la $L(G_{IC})$:

1. $w \in \Sigma^*$, $G \to G_{IC}$, $w \in L(G)$?
2. $G \to G_{IC}$, $L(G) = \emptyset$?

Obs: L(apd) automat push-down/ L(g1) = l(g2)
Deriv de lung 2 w – 1
Sirul vid /

Obs:

1. Ac. întrebări sunt valabile și pt. $L(APD)$ (APD poate fi obținut din GIC)
   ↓ autom. push-down
2. Lipsa întrebărilor de forma $L(G_1) = L(G_2)$ nu este întâmplătoare => nu pot fi sol. alg.

dem:

1. Analizez toate deriv. până la o anumită lung. / toți arb. de derivare până la o anum. h.

Fie $G = (V, \Sigma, R, S) \to G_{IC}$

Dacă (pt. w) regulile sunt numai de forma $A \to u$, $u \in \Sigma$ sau $|u| \geq 2$, atunci este suficient să inspectez deriv. de lung. $\leq 2|w| - 1$ / arb. de derivare de înălțime $|w|$

Elimin → $A \to e$, $A \to B$, $A, B \in V - \Sigma$

Dacă $L(G)$ conține șirul vid e, atunci trebuie să păstrez $S \to e$

Elim $A \to e$

$A \to e$
$B \to uAv$ | → adaug $B \to uv$, $u, v \in V^*$

Se repetă până când nu se mai pot adăuga noi reguli.
La sfârșit se elimină $A \to e$.

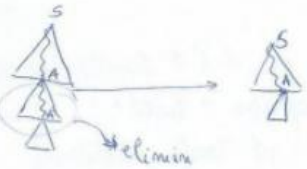
Elim. $A \to B$

$A \to B$
$B \to u$ | → adaug $A \to u$

Procesul se repetă pt. toate regulile
La sf. se elim. $A \to B$

2. $L(G) = \emptyset$?

Orice șir din $L(G)$ va avea un arbore de derivare.



=> e suficient să inspectez arb. de derivare de $h \leq |V - \Sigma|$

Tehnici de constructie a GIC

Deschideți cu

1° a^*, $a \in \Sigma$, $A \to aA \mid e$

2° $a^n b^n$, $a, b \in \Sigma$, $n \geq 0$, $X \to aXb \mid e$

3° $a^n b^{3n}$, $a, b \in \Sigma$, $n \geq 0$, $X \to aXbbb \mid e$

4° $a^m b^m$, $a, b \in \Sigma$, $m > n \geq 0$

$a^n b^n b^k$

$S \to XY$
$X \to aXb \mid e$
$Y \to bY \mid b$

5° $a^n b^{nr}$, $n \neq nr$

$L = L_1 \cup L_2$
$L_1 = \{a^n b^m \mid n > m\}$
$L_2 = \{a^n b^m \mid n < m\}$

6° $a^n b^m c^m d^n$

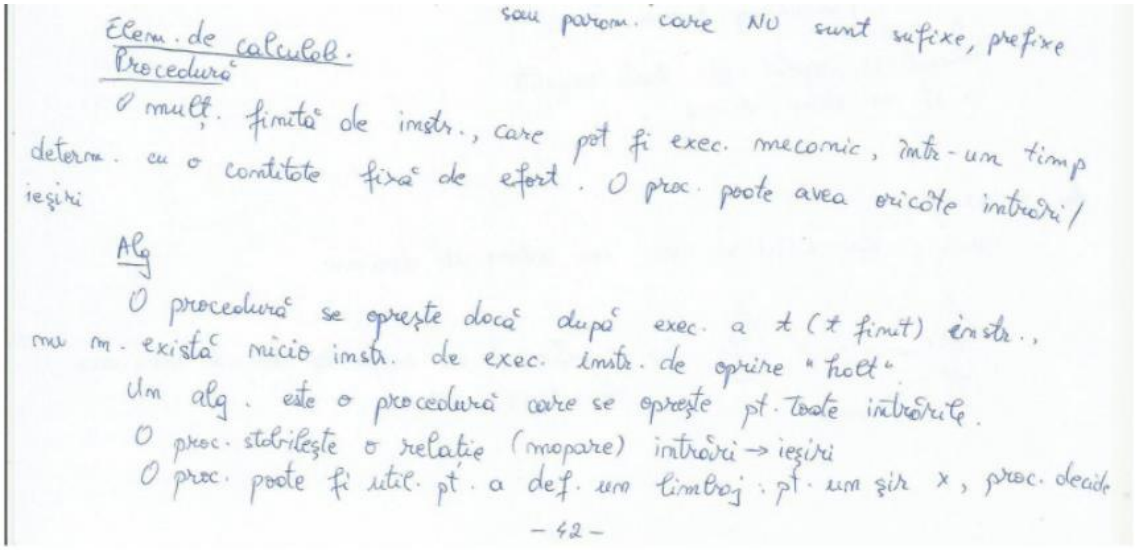
$S \to aSd \mid X$
$X \to bXc \mid e$

Nu se poate cu GIC: 1° să controlezi m. mult de 2 parom.
$a^m b^n c^m$ LIC

2° $a^n b^m c^n d^m$
sau parom. care NU sunt sufixe, prefixe

Elem. de calculab.
Procedură

Elem de calculi
Procedura/ efort/ instructiune de oprire halt / un algoritm este o procedura/ mapare/ utilizare/ recursiv enumerabila / recursiva



sau parcurs. care NU sunt sufixe, prefixe

Elem. de calculab.

Procedură

O mulț. finită de instr., care pot fi exec. mecanic, într-un timp determ. cu o cantitate fixă de efort. O proc. poate avea oricâte intrări/ ieșiri

Alg

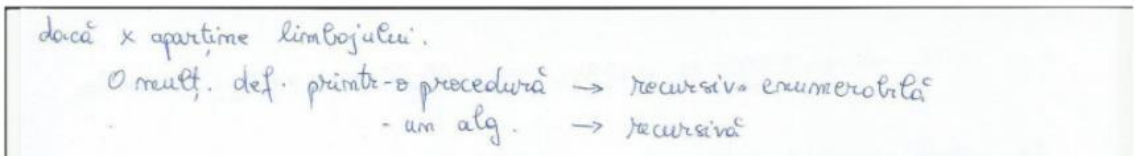
O procedură se oprește dacă după exec. a t (t finit) instr., nu mai există nicio instr. de exec. instr. de oprire "halt"

Un alg. este o procedură care se oprește pt. toate intrările.

O proc. stabilește o relație (mapare) intrări → ieșiri

O proc. poate fi utilă pt. a def. un limbaj: pt. un șir x, proc. decide

– 42 –



dacă x aparține limbajului.

O mulț. def. printr-o procedură → recursiv enumerabilă

- un alg. → recursivă

Formalizarea notiunii de procedra
M turing
Ierarhia chausky, markov, lambda/ taj/ post/ gramatici regulate/ gic/ gdc (gramatici dependente de context) / gramatici nerestrictionate

Formalizarea noțiunii de procedură

→ M. Turing (1936 - 1937)
→ Ierarhia Chomsky (1959, 1963)
→ Alg. Markov (1951)
→ Calc. Lambda (1941)
→ Sist. Tag / Post (~ 1950)

Toate formalism. sunt echiv. atât cât al calculabilității.

Ierarhia Chomsky

1) Gramatici regulate
$A \to xB \mid x$, $x \in \Sigma^*$, $A, B \in V - \Sigma$

2) GIC
$A \to \alpha$, $A \in V - \Sigma$, $\alpha \in V^*$

3) GDC (gram. dependente de context)
$\alpha A \beta \to \alpha \sigma \beta$, $|\alpha A \beta| \leq |\alpha \sigma \beta|$, $A \in V - \Sigma$, $\alpha, \beta, \sigma \in V^*$

4) Gramatici nerestricționate (GO) (= zero)
$V^*(V-\Sigma)V^* \to V^*$

Pt. fiecare tip de gramatică ∃ acceptare.

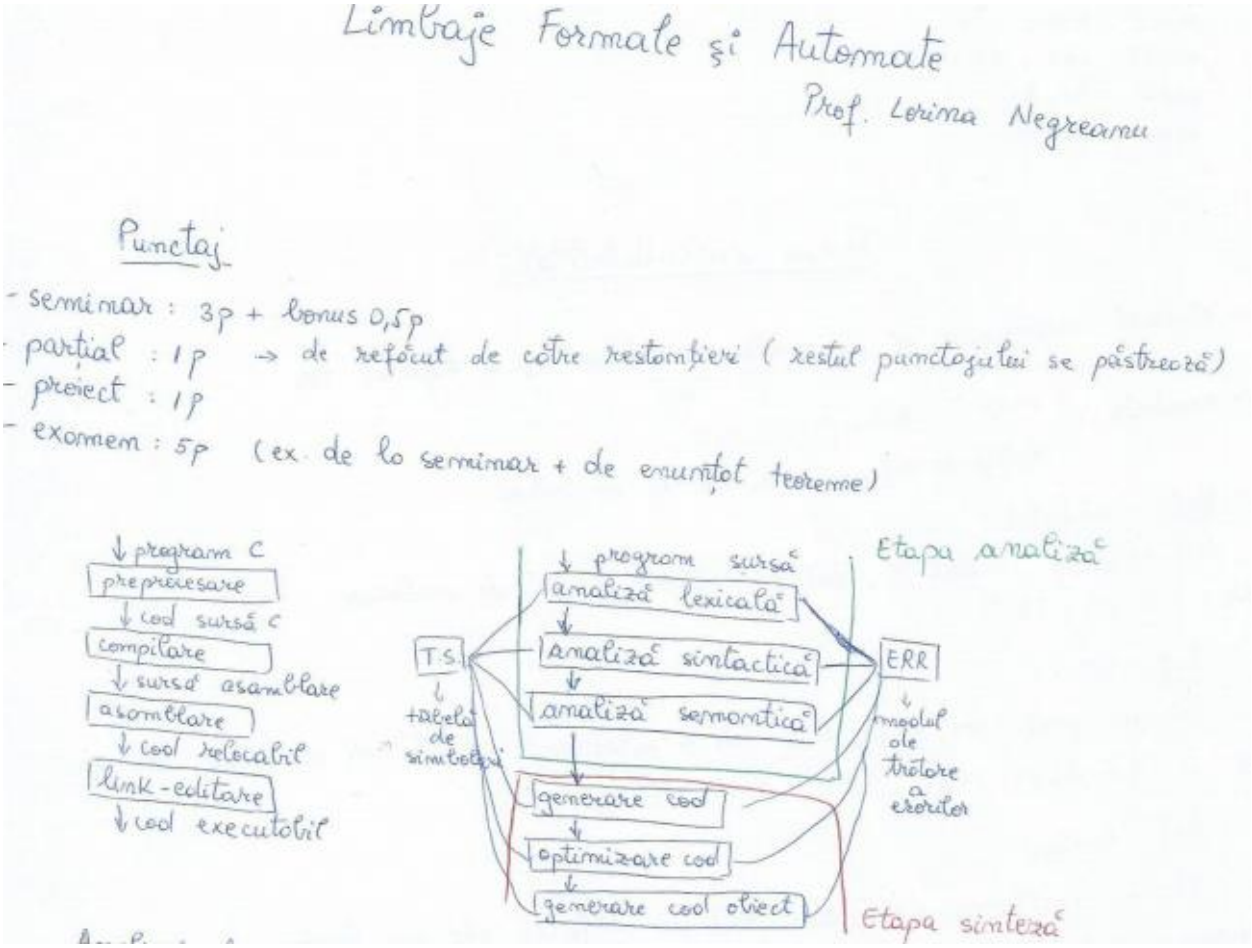
1) LR → AF
2) LIC → APD (nedeterm.)
3) LDC → 2 way nedeterm. linear bounded automatic
4) L. Nerestricționate → M. Turing

ex GDC → $L = \{ a^m b^m c^m \mid m > 0 \}$

$S \to aSBC \mid abC$
$CB \to BC$
$bB \to bb$
$cC \to cc$
$bC \to bc$

$S \Rightarrow aSBC \Rightarrow aabCBC \Rightarrow aabBCC \Rightarrow aabbCC \Rightarrow aabbcC \Rightarrow aabbcc$

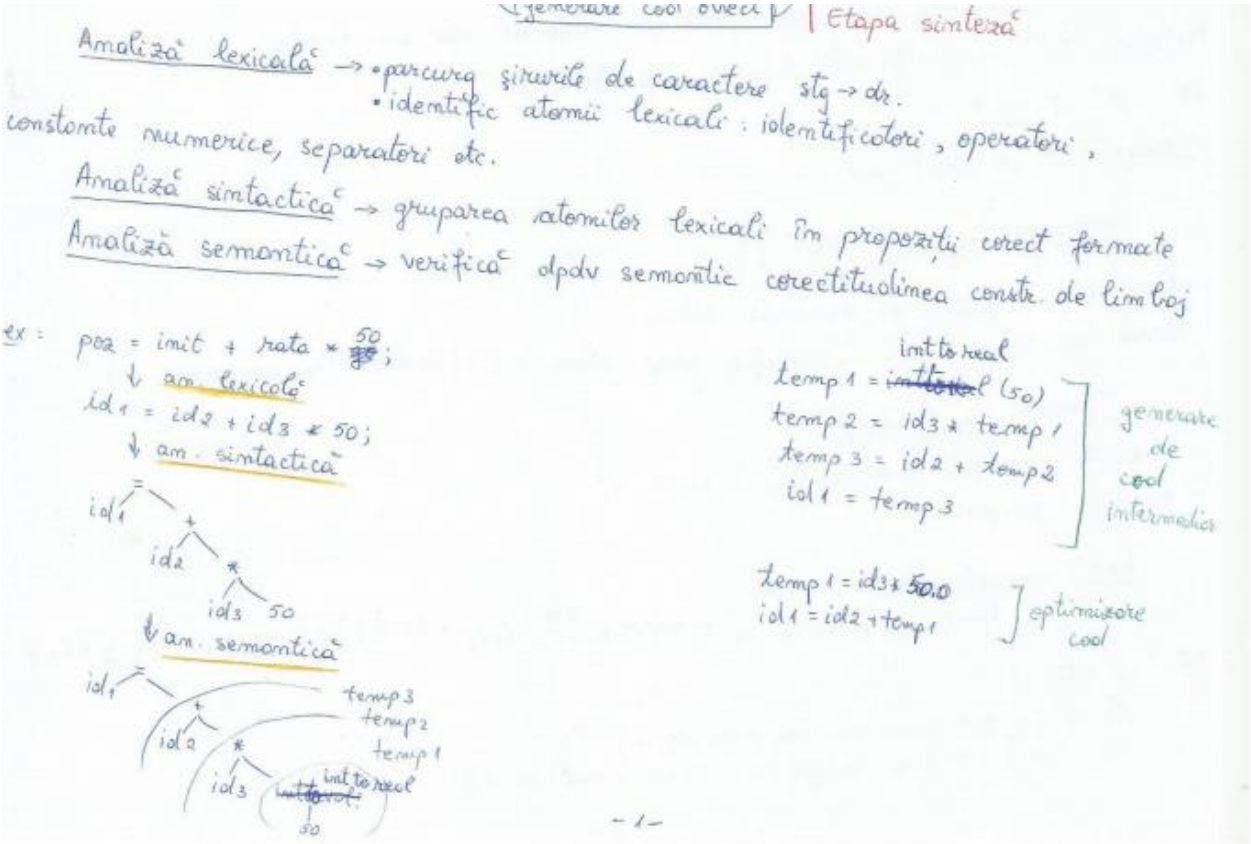
LFA CUPRINS



Analiza lexicala

Analiza sintactica

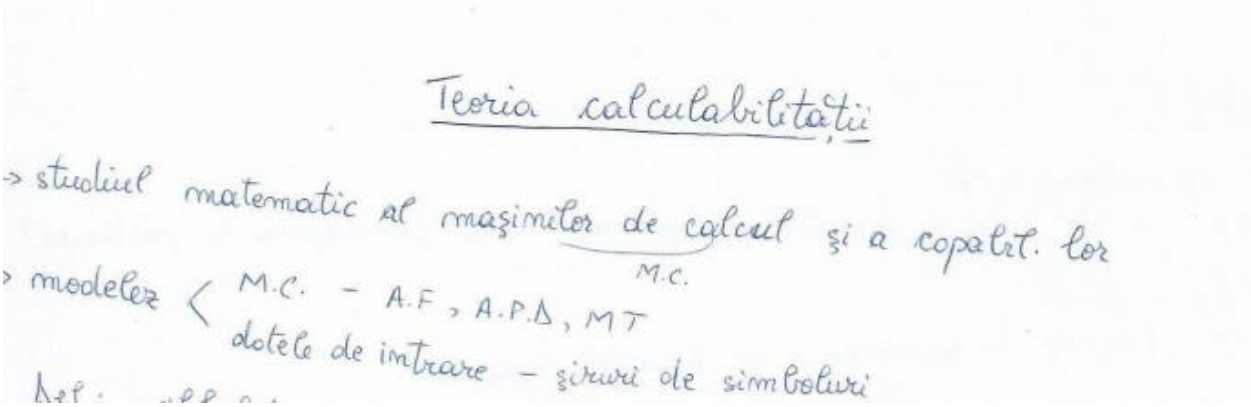
Analiza semnatica



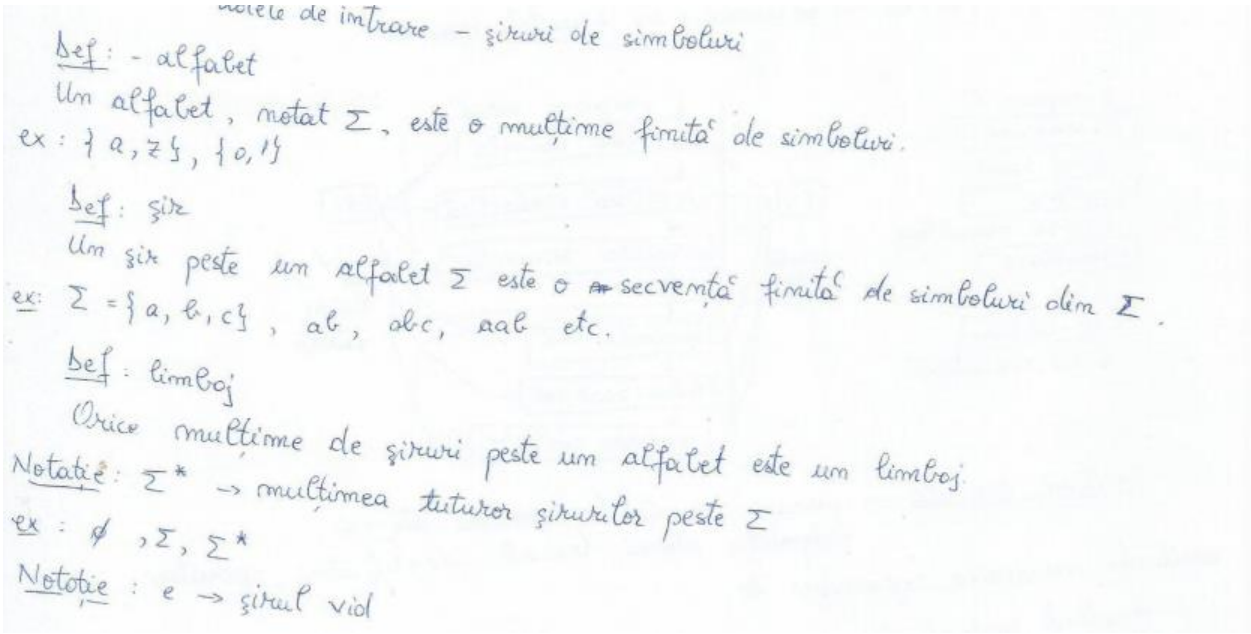

```
MOVF   id3, R1
MULF   #50.0, R1
MOVF   id2, R2
ADDF   R2, R1
MOVF   R1, id1
```

Teoria calculabilitatii

- Studiul matematic al masinilor de calcul
- Modelez -MC – AF, APD, MT
- Datele de intrare : siruri de simboluri

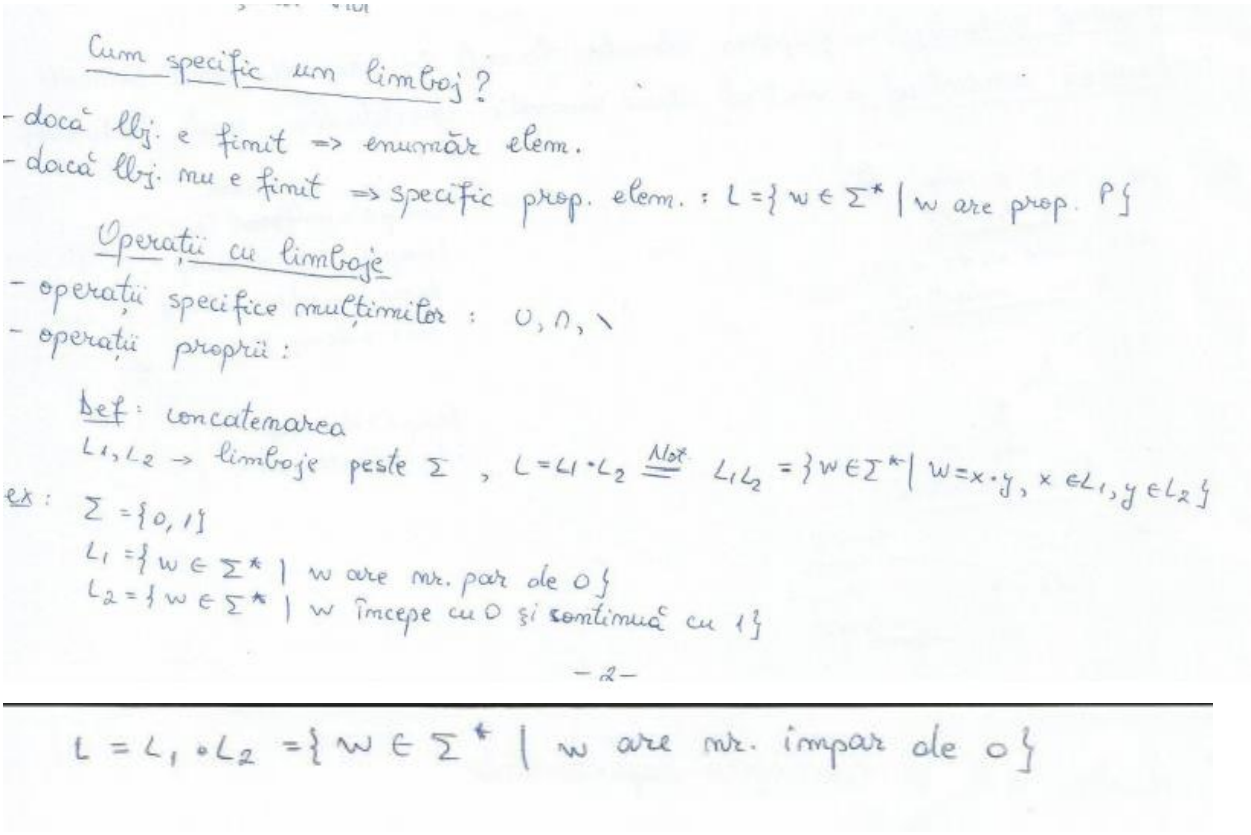


Def : alfabet, sir, limbaj



Limbaj finit/ nu e finit
Operatii cu limbaje:

- Concatenarea :



Cum specific un limbaj ?

- dacă lbj. e finit => enumăr elem.
- dacă lbj. nu e finit => specific prop. elem. : $L = \{ w \in \Sigma^* \mid w \text{ are prop. } P \}$

Operații cu limbaje

- operații specifice mulțimilor : $\cup, \cap, \setminus$
- operații proprii :

Def: concatenarea

$L_1, L_2 \to$ limbaje peste Σ, $L = L_1 \cdot L_2 \overset{\text{not}}{=\!=} L_1 L_2 = \{ w \in \Sigma^* \mid w = x \cdot y, x \in L_1, y \in L_2 \}$

ex: $\Sigma = \{0, 1\}$

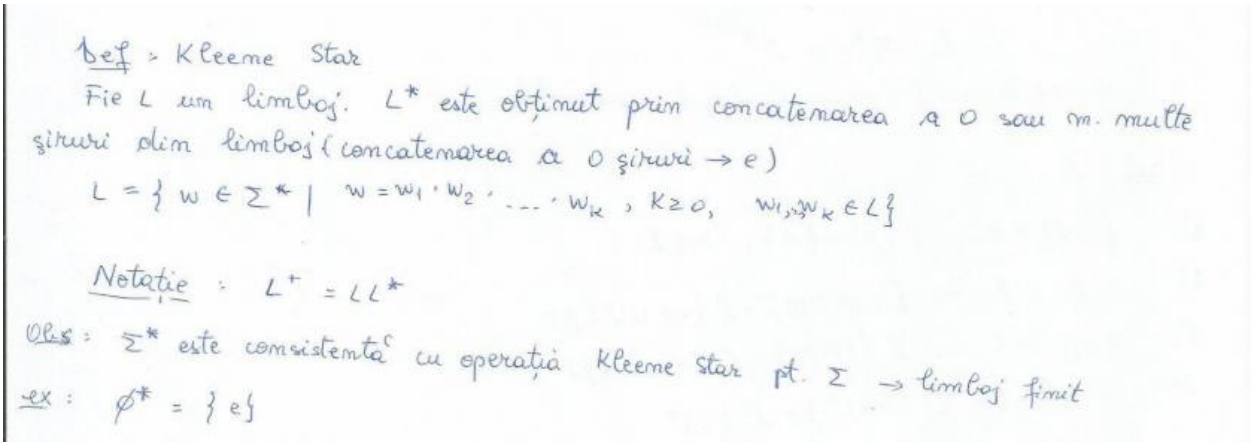
$L_1 = \{ w \in \Sigma^* \mid w \text{ are nr. par de } 0 \}$

$L_2 = \{ w \in \Sigma^* \mid w \text{ începe cu } 0 \text{ și continuă cu } 1 \}$

– 2 –

$L = L_1 \circ L_2 = \{ w \in \Sigma^* \mid w \text{ are nr. impar de } 0 \}$

Kleene Star : Fie L un limbaj. L* este obtinut prin concatenarea a 0 sau mai multe siruri din limbaj. Notatie.



Def: Kleene Star

Fie L un limbaj. L^* este obținut prin concatenarea a 0 sau nr. multe șiruri din limbaj (concatenarea a 0 șiruri $\to \varepsilon$)

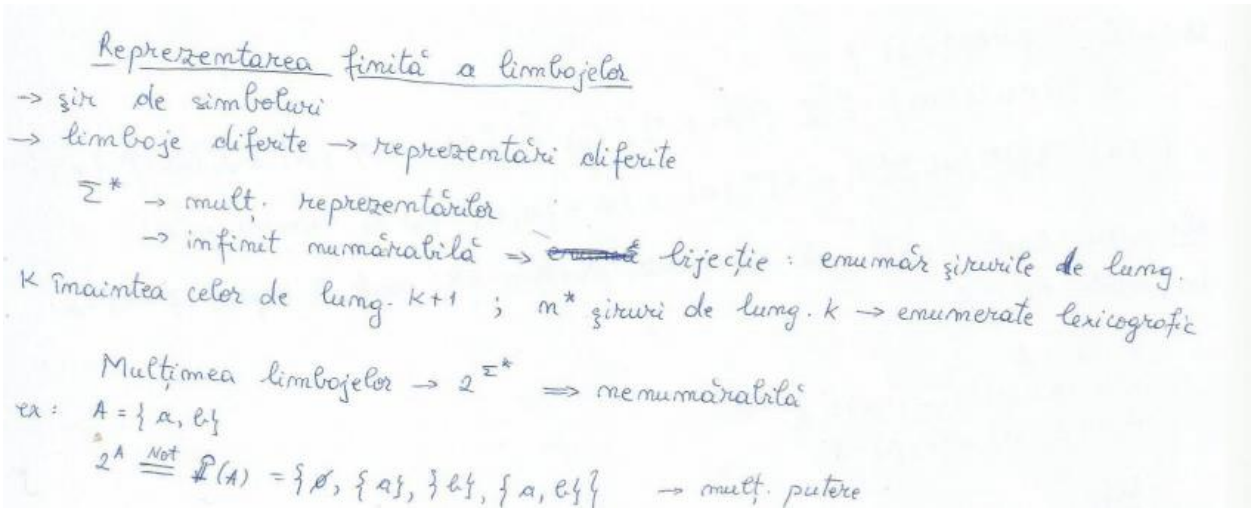
$L = \{ w \in \Sigma^* \mid w = w_1 \cdot w_2 \cdot \ldots \cdot w_k, k \geq 0, w_1, \ldots, w_k \in L \}$

Notație: $L^+ = LL^*$

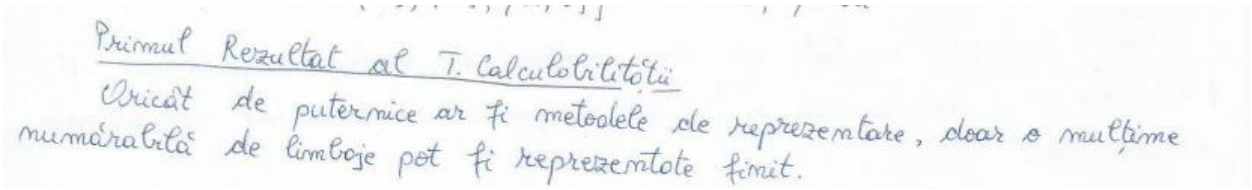
Obs: Σ^* este consistentă cu operația Kleene star pt. $\Sigma \to$ limbaj finit

ex: $\emptyset^* = \{\varepsilon\}$

Reprezentarea finita a limbajelor:
-sir de simboluri
-limbaje diferite -> reprezentari diferite
-multimea reprezentarilor
-numarabila / nenumarabila / bijectie / lexicografic

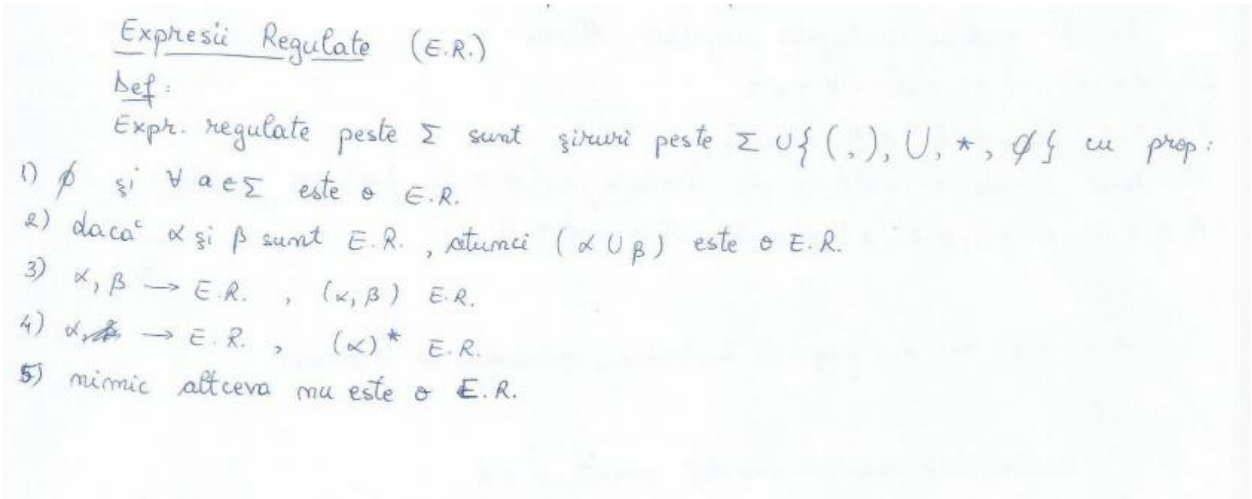


Primul rezultat al Teoremei Calculabilitatii
Oricat de puternice ar fi metodele de reprezentare, doar o multime numarabila de limbaje pot fi reprezentate finit.

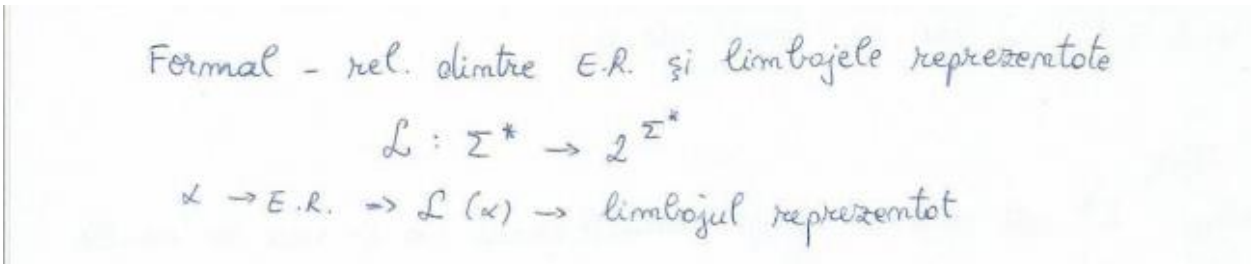


EXPRESII REGULATE

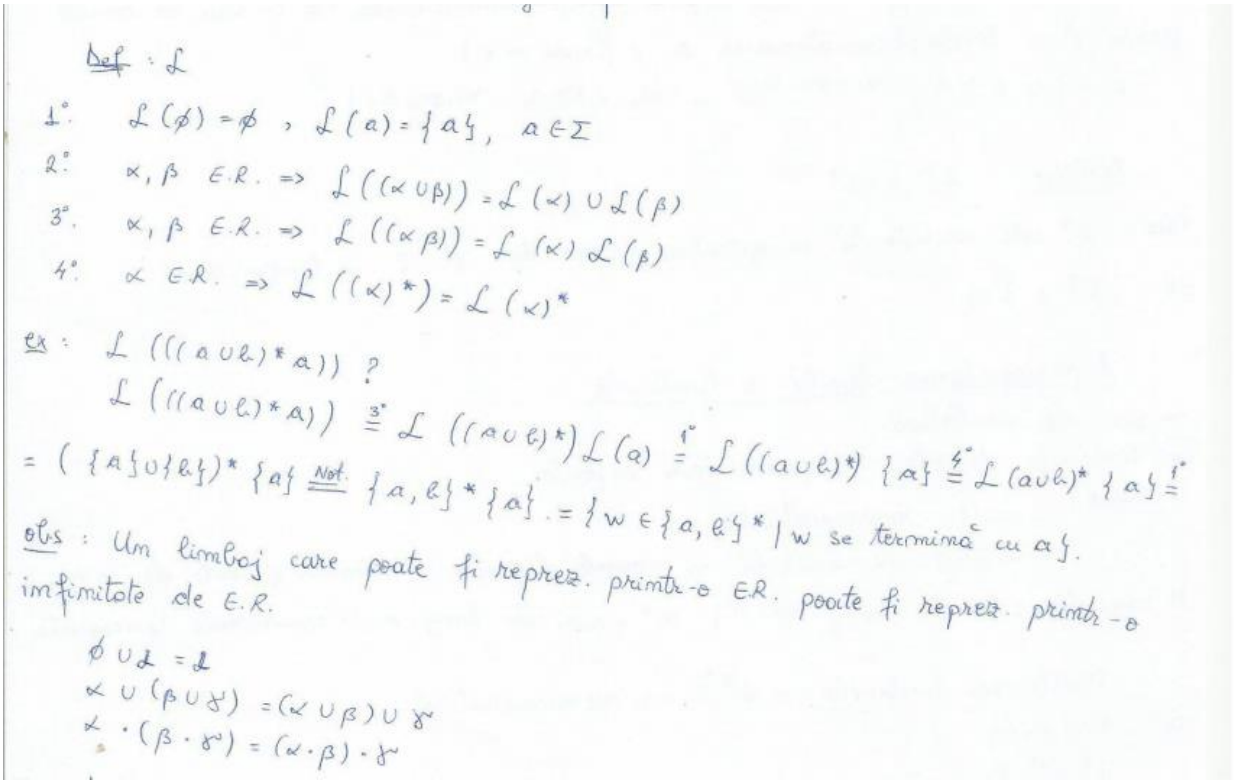
Proprietati:



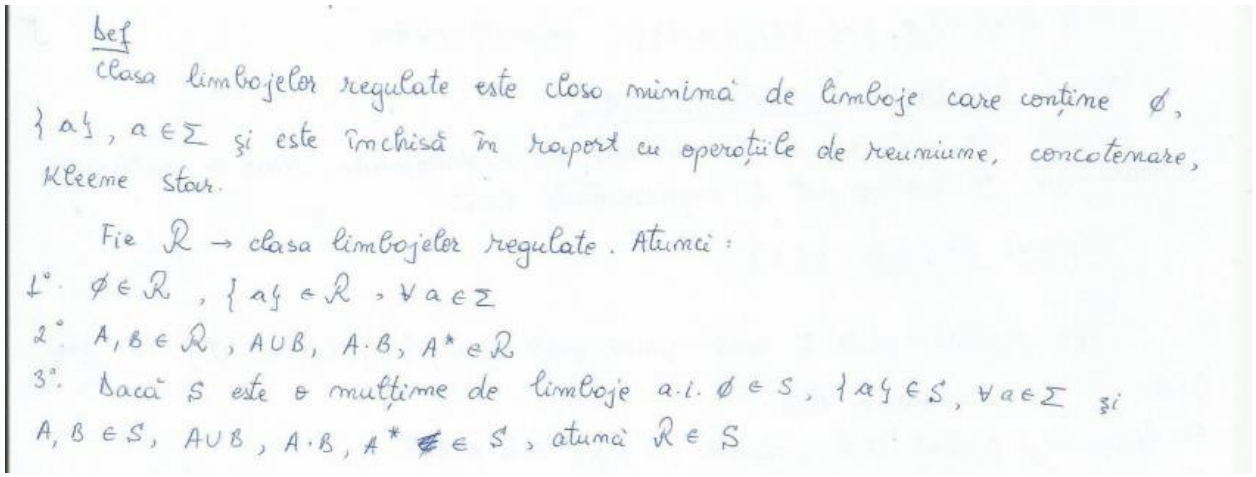
Relatia dintre expresii regulate / ER si limbaje reprezentate :



Def : expresii regulate / ER
Un limbaj care poate fi reprezentat printr-o expresie regulata, poate fi reprezentat printr-o infinitate de expresii regulate.



Def :
Clasa limbajelor regulate este clasa minima de limbaje si este inchisa in raport cu operatiile de reunine, concatenare, Kleene Star.
Clasa limbajelor regulate :



Def
Clasa limbajelor regulate este cea mai mica clasa de limbaje care contine $\emptyset$, $\{a\}$, $a \in \Sigma$ si este inchisa in raport cu operatiile de reuniune, concatenare, Kleene star.

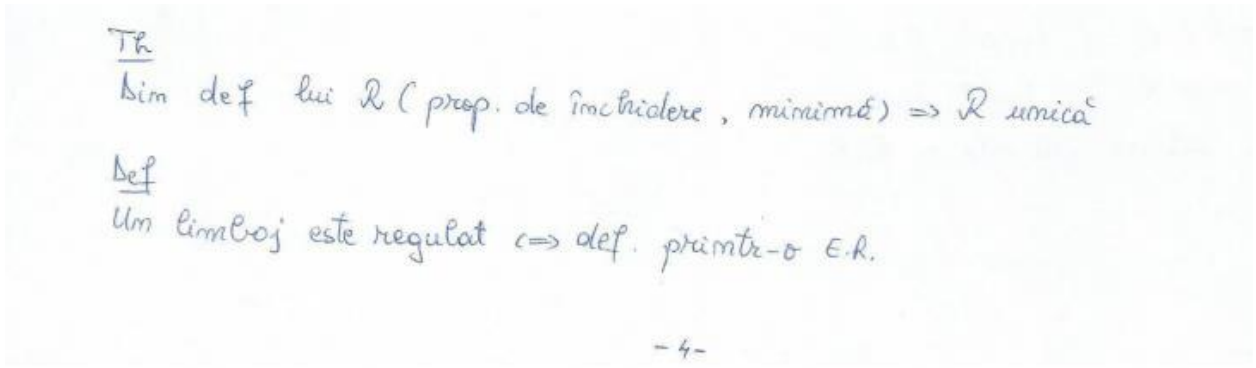
Fie $\mathcal{R}$ → clasa limbajelor regulate. Atunci:

1° $\emptyset \in \mathcal{R}$, $\{a\} \in \mathcal{R}$, $\forall a \in \Sigma$

2° $A, B \in \mathcal{R}$, $A \cup B$, $A \cdot B$, $A^* \in \mathcal{R}$

3° Daca S este o multime de limbaje a.i. $\emptyset \in S$, $\{a\} \in S$, $\forall a \in \Sigma$ si $A, B \in S$, $A \cup B$, $A \cdot B$, $A^* \in S$, atunci $\mathcal{R} \in S$

Teorema : Din definitia lui R (proprietate de inchidere , minima) => R unica
Def : Un limbaj este regulat daca si numai daca este definit printr-o expresie regulata

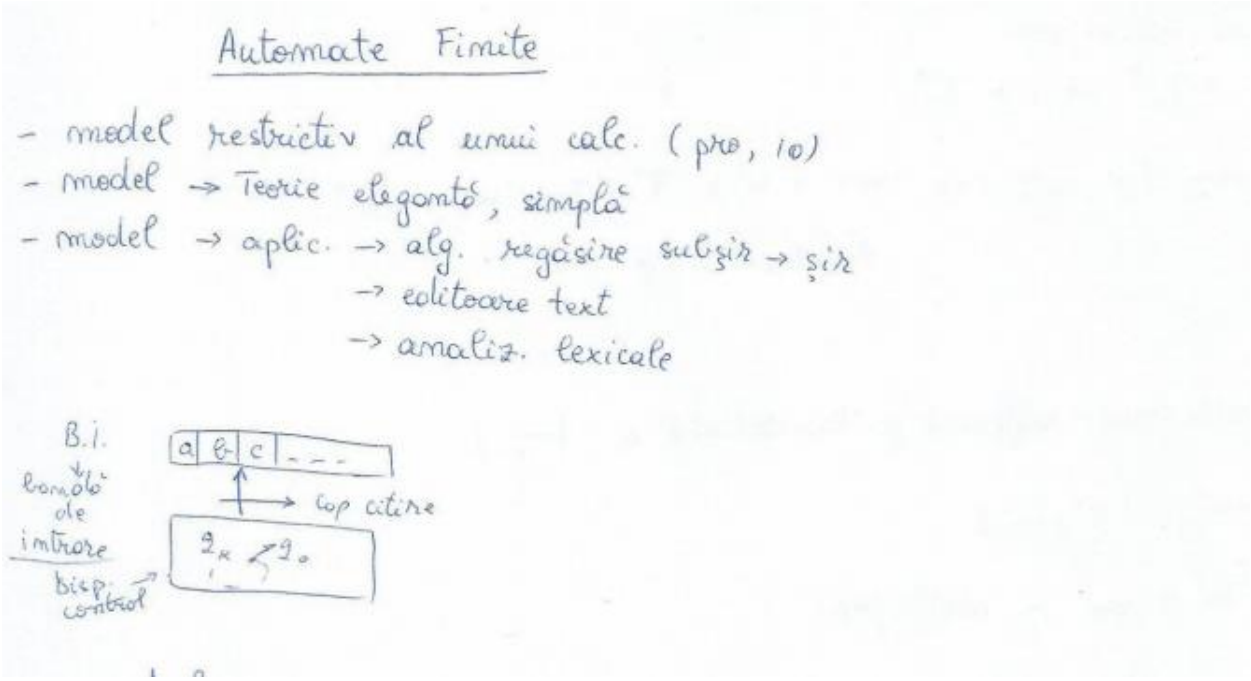


Th
Din def lui $\mathcal{R}$ (prop. de inchidere, minimă) => $\mathcal{R}$ unică

Def
Un limbaj este regulat <=> def. printr-o E.R.

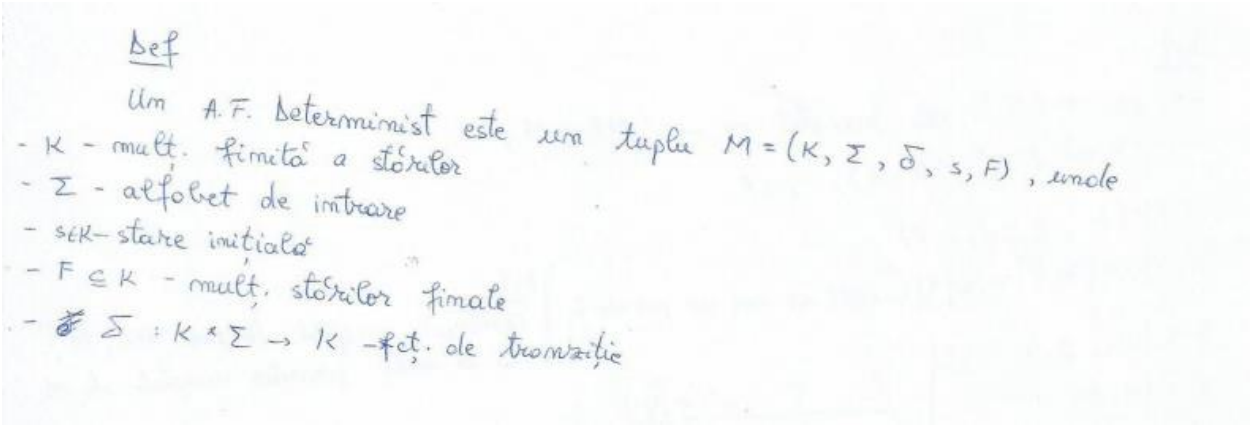
AUTOMATE FINITE :

- Model restrictiv al unui calc
- Model : teorie eleganta/simpla
- Editoare text/ analiza lexicala/ regasire sir/ subsir



Un automat determinist este un tuplu:

- Multime finita a starilor
- Alfabet de intrare
- Stare initiala
- Multimea starilor finale
- Fct de tranzitie

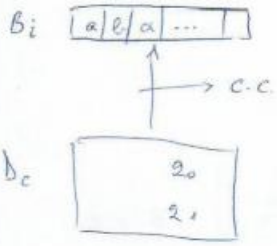


Configuratia unui AFD / Automat finit determinist :

12.10.2016

Curs 2

Automate finite deterministe



$M = (K, \Sigma, \delta, s, F)$

$K \rightarrow$ mult. finită de stări

$\Sigma \rightarrow$ alfabet

$s \in K \rightarrow$ st. inițială

$F \subseteq K \rightarrow$ mult. st. finale

$\delta : K \times \Sigma \rightarrow K$ fct. tranziție

Def:

Configurația unui AFD, $M = (K, \Sigma, \delta, s, F)$, este un elem. din $K \times \Sigma^*$

-trecerea intr-un pas

-inchiderea reflexiva

- un sir este acceptat de un AFD

Def trecere într-un pas

$\vdash_M : K \times \Sigma^* \to K \times \Sigma^*$

$(q, w) \vdash_M (q', w') \iff w = \tau w', \ \tau \in \Sigma, \ w, w' \in \Sigma^*$

$\delta(q, \tau) = q'$

Def

$\vdash_M^*$ (închiderea reflexivă și tranzitivă a $\vdash_M$)

$(q, w) \vdash_M^* (q', w')$

↑ în 0 sau m. multi pași

Def

Un șir $w \in \Sigma^*$ este acceptat de un AFD M,

$(s, w) \vdash_M^* (q, e)$, $q \in F$

ex: ? $M = (K, \Sigma, \delta, s, F)$

$L(M) = \{w \in \{a, b\}^* \mid w$ conține un nr. par de $b\}$

$K = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

$s = q_0$

$F = \{q_0\}$

q	τ	$\delta(q, \tau)$
q_0	a	q_0
q_0	b	q_1
q_1	a	q_1
q_1	b	q_0

Def

Limbajul acceptat de un AFD, M, este mult. șirurilor acceptate de M.

Activati Windows

Reprezentare AFD prin diagrama de tranzitie : graf directionat :

Reprezentare AFA prin diagr. de tranziție → graf orientat
→ noduri → stări
→ arce → tranziții
→ st. init. >0
→ st. finală ◎

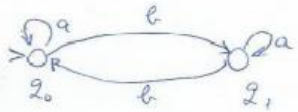


abb : $>q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1 \xrightarrow{b} q_0 \in F$

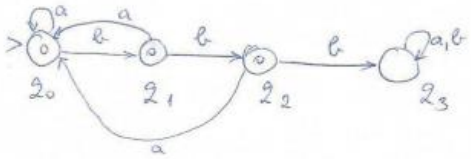
ab : $>q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_1 \notin F$

→ pt. că nu ajung într-o stare finală

Ex : w nu contine 3 b consecutive : trb sa am in fiecare stare tranzitii dif pt toate simbolurile :

ex. ? $M = (K, \Sigma, \delta, s, F)$

$L_{\neq}(M) = \{ w \in \{a, b\}^* \mid w$ nu conține 3 b consecutive $\}$



→ trb. să am în fiecare stare tranziții def. pt. toate simbolurile

$K = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$s = q_0$

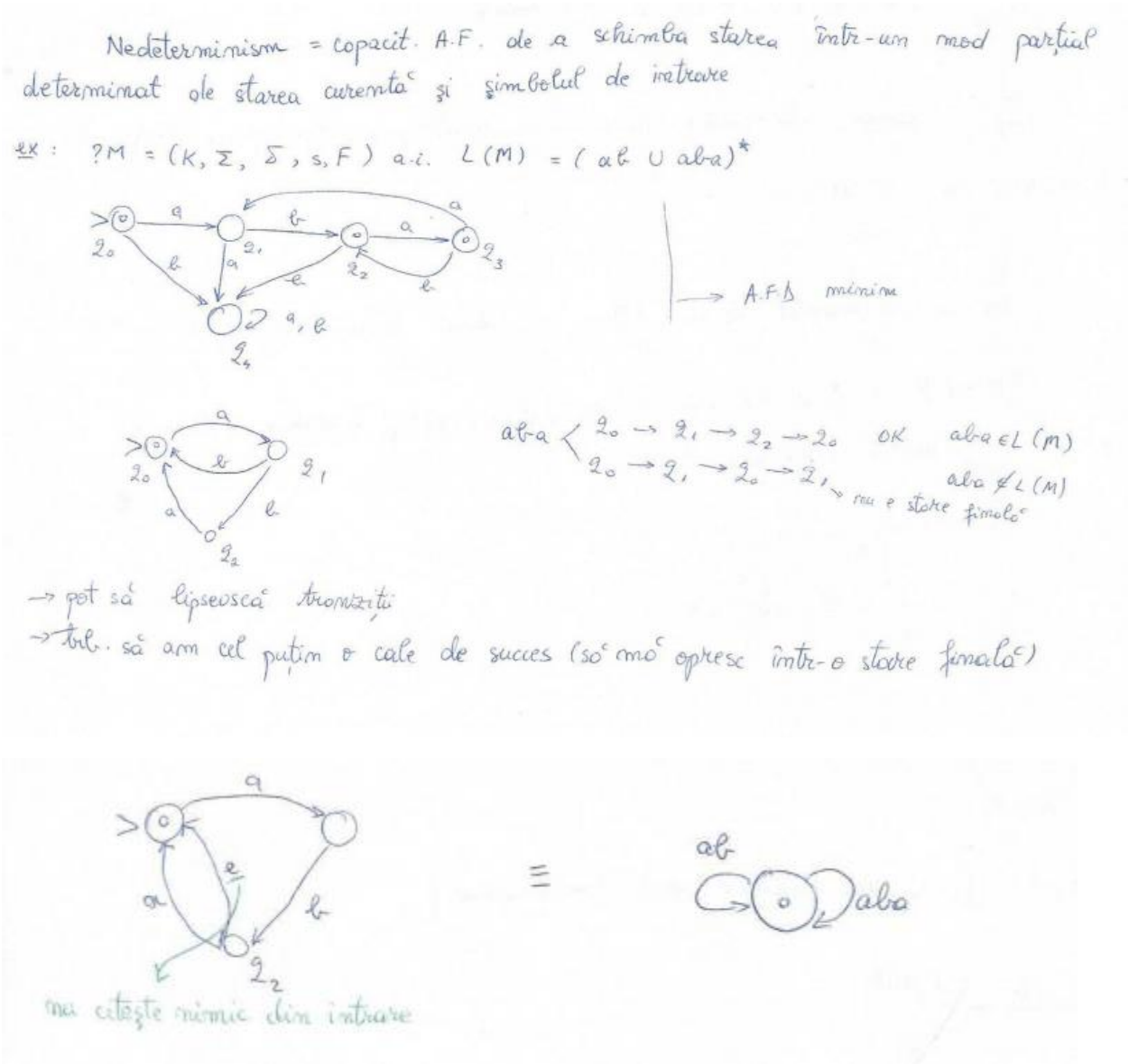
$F = \{q_0, q_1, q_2\}$

δ → def. prin diagrame de tranziție

AUTOMATE FINITE NEDETERMINISTE :

NEDETERMINISM = capacitatea unui Automat Finit de a schimba starea intr-un mod partial determinat de starea curenta si simbolul de intare :

- Pot sa lipseasca tranzitii
- Trb sa am cel putin o cale de succes (sa ma opresc intr-o stare finala)



DEF : Un automat finit nedetrminist / AFN este un tuplu

Configuratie AFN

RELATIE

Trecerea intre configuratii in 0 sau mai multi pasi (inchiderea refexiva si tranzitiva)

Def

Un automat finit nedeterminist este un tuplu $M = (K, \Sigma, \Delta, s, F)$

$K \rightarrow$ mult. finită a stărilor

$\Sigma \rightarrow$ alfabet

$s \in K \rightarrow$ st. inițială

$F \subseteq K \rightarrow$ mult. st. finale

$\Delta \subseteq K \times \Sigma^* \times K$

↑ submult. finită

Def

O config. a unui AFN este un elem. din $K \times \Sigma^*$.

Def

$\vdash_M : (K \times \Sigma^*) \times (K \times \Sigma^*) \rightarrow$ relație

Def

$\vdash_M^*$ Trecerea între config. în 0 sau mai multi pași (închiderea reflex. și tranzitivă a $\vdash_M$)

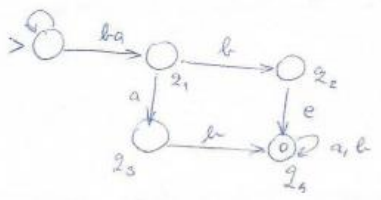
Un sir este acceptat de un AFN daca :

Ex : w contine cel putin o aparitie a subsirului bab sau baab

Def

Un șir este acceptat de un AFN, M, dacă $(s, w) \vdash_M^* (q, e)$, $q \in F$

ex : $M = (K, \Sigma, \Delta, s, F)$ a.î. $L(M) = \{ w \in \{a, b\}^* \mid w$ conține cel puțin o apariție a subșirului bab sau baab

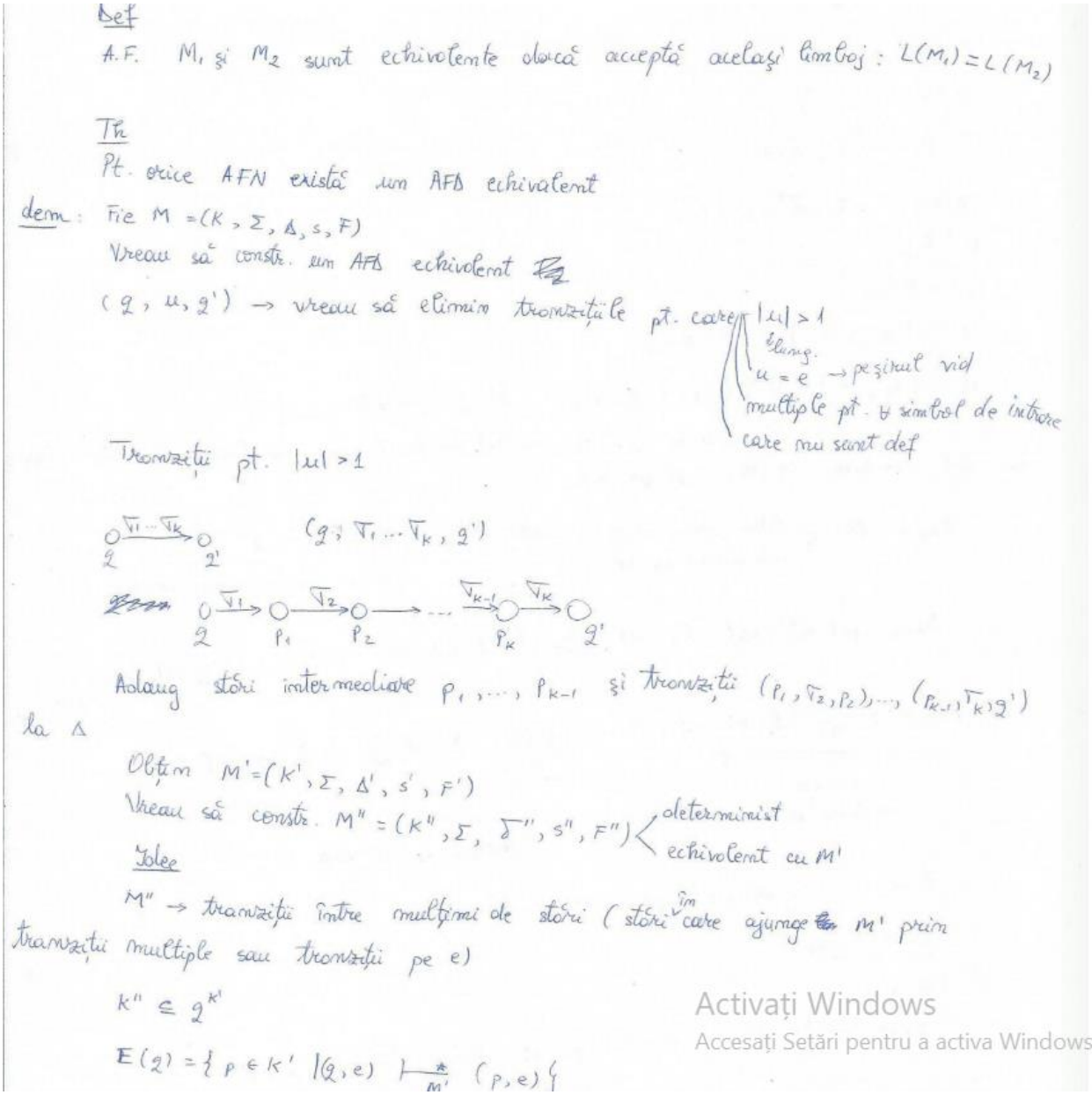


ECHIVALENTA AFN SI AFD

Def : 2 automate finite sunt echivalente daca accepta acelasi limabj

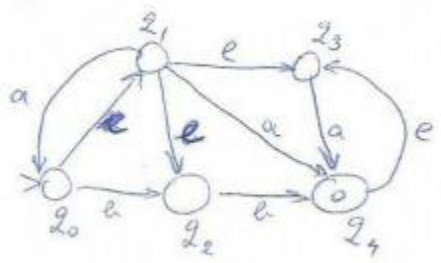
Th : Pt orice AFN exista un AFD echivalent

Dem : Tranzitii / stari intermediare



Ex :

ex



$E(q_0) = \{q_0, q_1, q_2, q_3\}$

$E(q_1) = \{q_1, q_2, q_3\}$

$E(q_2) = \{q_2\}$

$E(q_3) = \{q_3\}$

$E(q_4) = \{q_3, q_4\}$

$M'' = (K'', \Sigma, \delta'', s'', F'')$

$K'' \subseteq 2^{K'}$

$s'' = E(s')$

$F'' = \{Q \subseteq K' \mid Q \cap F' \neq \emptyset\}$

$\delta''(Q, \nabla) = \bigcup \{E(p) \mid (q, \nabla, p) \in \Delta', q \in Q, \nabla \in \Sigma\}$

Pt. fiecare q din Q, mă uit la tranz. care mă duce într-o stare, i.e. la închiderea cu tranz. pe ε a lui

Arăt M'' < determinist → dim constr. $\delta'' \rightarrow \emptyset \in K'' \subseteq 2^{K'}$
echivalent cu M' ↳ starea vidă e o stare validă

INDUCTIE

Pas de baza

Ipoteza inductiva

Dacă pot să arăt: $(q, w) \vdash_{M'}^{*} (p, e) \iff (E(q), w) \vdash_{M''}^{*} (P, e)$, $p \in P$

$(s, w) \vdash_{M'}^{*} (f, e)$, $f \in F \iff (E(s), w) \vdash_{M''}^{*} (P, e)$, $f \in P$

$w \in L(M')$ acceptarea pt. AFN

$w \in L(M'')$ acceptarea pt AFD

Dem. prin inducție după $|w|$

Pas de bază

$|w| = 0$

$(q, e) \vdash_{M'}^{*} (p, e) \iff (E(q), e) \vdash_{M''}^{*} (P, e)$, $p \in P$

$p \in E(q)$

s'', P''

$w \in L(M'')$

Ip. inductivă: PP adev pt $|w| = k$

Pas de inducție

$|w| = k+1$, $w = v\,a$, $v \in \Sigma^*$, $a \in \Sigma$

$(q, w) \vdash_M^* (p, e) \iff \exists\, r_1, r_2 \in K'$ a.i. $(q, va) \vdash_{M'}^* (r_1, a) \vdash_{M'} (r_2, e) \vdash_{M'}^* (p, e)$

I $(q, va) \vdash_{M'}^* (p, e) \Rightarrow (E(q), va) \vdash_{M''}^* (P, e)$, $p \in P$

$(q, va) \vdash_{M'}^* (r_1, a) \vdash_{M'} (r_2, e) \vdash_{M'}^* (p, e)$

$(q, v) \vdash_{M'}^* (r_1, e)$, $|v| = K$ $\xrightarrow{\text{ip. inductivă}}$ $(E(q), v) \vdash_{M''}^* (R_1, e)$, $r_1 \in R_1$

$(r_1, a) \vdash_{M'} (r_2, e) \Rightarrow (r_1, a, r_2) \in \Delta'$

dim constr. δ''

$\Rightarrow$ $E(r_2) \subseteq \delta''(r_1, a)$ | $\delta''(Q, \nabla) = \cup \{ E(p) \mid (q, a, p) \in \Delta' \}$
$q \in Q$, $\nabla \in \Sigma$, $p \in K'$

$(r_2, e) \vdash_{M'}^* \Rightarrow$ $p \in E(r_2)$

$E(r_2) \subseteq \delta''(R_1, a)$ | $\Rightarrow p \in \delta''(R_1, a) \Rightarrow (R_1, a) \vdash_{M''} (P, e)$, $p \in P$

$\Rightarrow (E(q), va) \vdash_{M''}^* (R_1, a) \vdash_{M''} (P, e)$, $p \in P$

II $\Leftarrow (E(q), va) \vdash_{M''}^* (R_1, a) \vdash_{M''} (P, e)$, $p \in P \Rightarrow (q, va) \vdash_{M'}^* (r_1, a)$

TEMĂ (pași invers echiv.) → ole acelas data viitoare la curs

$s'' = E(s') = E(q_0) = \{q_0, q_1, q_2, q_3\} = Q_0$

$s''(Q_0, a) = E(q_0) \cup E(q_4) =$ → care stări au tranz. pe a

$= \{q_0, q_1, q_2, q_3, q_4\} = Q_1$

$s''(Q_0, b) = E(q_2) \cup E(q_4) = \{q_2, q_3, q_4\} = Q_2$

$\delta''(Q_1, a) = Q_1$

$\delta''(Q_1, b) = Q_2$

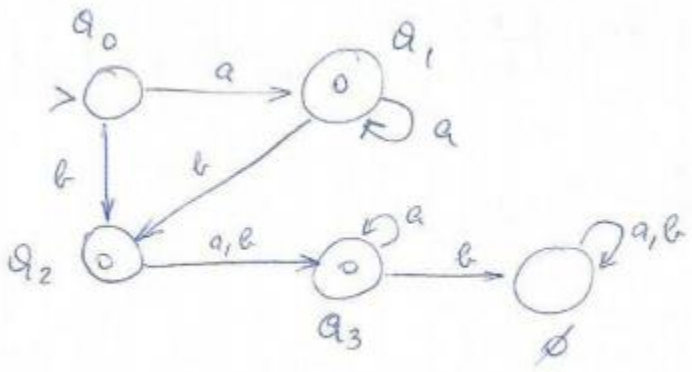
$\delta''(Q_2, a) = E(q_4) = \{q_3, q_4\} = Q_3$ → doar q_4 face tranz. pe a

$\delta''(Q_2, b) = E(q_4) = Q_3$

$\delta''(Q_3, a) = Q_3$

$\delta''(Q_3, b) = \emptyset$

$s''(Q, T) = \cup \{E(p) \mid (q, T, p) \in \Delta', q \in Q, p \in K', T \in \Sigma\}$



Proprietati de inchidere ale L(AF)

Teorema : Clasa limbajelor acceptate de AF este inchisa in raport cu op de

-reuniune

- concatenare

- Kleene Star

-Complementare

-Intersectie

DEM:

Th

Clasa limbajelor acceptate de AF este închisă în raport cu op. de:

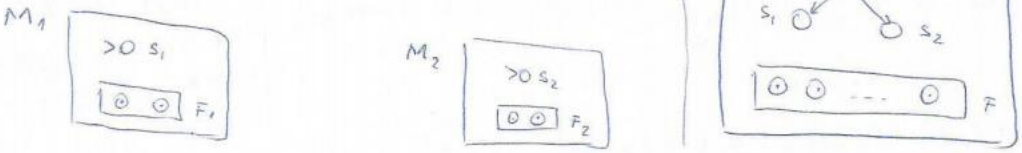
a) reuniune
b) concatenare
c) Kleene star
d) complementare
e) intersecție

dem:

a) $M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

$M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$

$?M, L(M) = L(M_1) \cup L(M_2)$



$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup K_2 \cup \{s\}$

$s = s$

$F = F_1 \cup F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup \{(s, e, s_1), (s, e, s_2)\}$

$w \in L(M) \iff (s, w) \vdash_M^* (f, e)$, $f \in F$

$\iff (s_1, w) \vdash_{M_1}^* (f, e)$, $f \in F_1$

sau

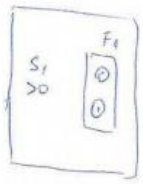
$(s_2, w) \vdash_{M_2}^* (f, e)$, $f \in F_2$

b) concatenare

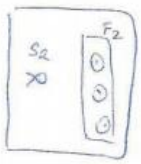
$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

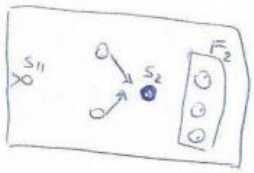
$M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$ $\longmapsto$? M a.i. $L(M) = L(M_1) \cdot L(M_2)$

M_1



M_2





$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup K_2$

$s = s_1$

$F = F_2$

$\Delta = \Delta_1 \cup \Delta_2 \cup (f_1 \times \{e\} \times \{s_2\})$

$(s, w) \vdash_M^* (f, e)$, $f \in F$

$\Updownarrow$

$w \in L(M)$

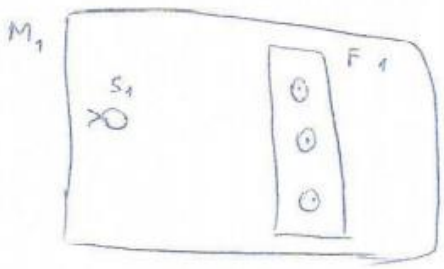
$\Leftrightarrow$ $\exists w_1, w_2 \in \Sigma^*$ a.i. $w = w_1 w_2$ și $p \in F_1$

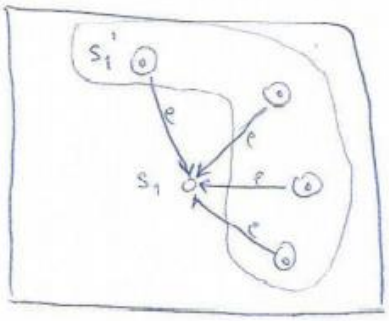
$(s_1, w_1) \vdash_{M_1}^* (p, e)$ și $(s_2, w_2) \vdash_{M_2}^* (f, e)$

c) Kleene Star

$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$

? M a.i. $L(M) = L(M_1)^*$





$M = (K, \Sigma, \Delta, s, F)$

$K = K_1 \cup \{s_1'\}$

$s = s_1'$

$F = F_1 \cup \{s_1'\}$

$\Delta = \Delta_1 \cup \{F_1 \times \{e\} \times \{s_1\} \cup \{(s_1', e, s_1)\}$

d) complementare

$M = (K, \Sigma, \delta, s, F)$ $\Rightarrow$ interschimb stările finale cu cele nefinale

? $\bar{M}$ a.i. $L(\bar{M}) = \Sigma^* - L(M)$

e) intersectie

M_1, M_2 A.F. , $L_1 = L(M_1)$, $L_2 = L(M_2)$

$L = L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$

AFD AFD

AFN

Convertesc AFN & AFD

sau

$M_1 = (K_1, \Sigma, \delta_1, s_1, F_1)$

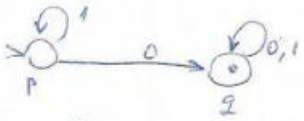
$M_2 = (K_2, \Sigma, \delta_2, s_2, F_2)$

?M a.i. $L(M) = L(M_1) \cap L(M_2)$

$M = (K_1 \times K_2, \Sigma, \delta, (s_1, s_2), F_1 \times F_2)$

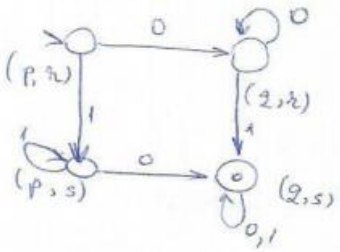
$\delta((p,q), a) = (\delta_1(p,a), \delta_2(q,a))$, $p \in K_1$, $q \in K_2$

ex : M_1 :



M_2 :





ECHIVALENTA LR = L(AF)

TEOREMA : Un limbaj este regulat daca si numai daca este accpetat de un automat finit.

DEM : LR / L AF / Clasa LR este clasa minima de limbaje / Ar ca L(M) este reuniunea unui nr finit de LR (R). / R este multimea sirurilor care duc automatul din starea qi in starea qj fara sa treaca printr-o stare intermediara.

Echivalenta L.R. ≡ L(A.F.)

$\{a, b\}^*$, $a^* b^*$

Th

Un limbaj este regulat (=> este ~~altceva~~ acceptat de un A.F.

demo

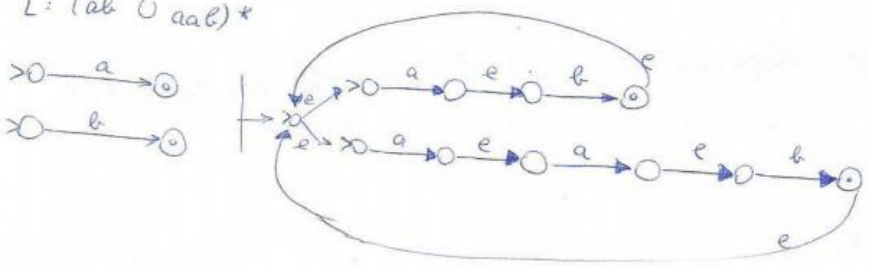
"=>" L.R. → L(A.F.)

Clasa L.R. este clasa minimă de limbaje care conține $\emptyset$, $\{\sigma\}$, $\forall \sigma \in \Sigma$ și este închisă în raport cu operațiile de $\cup$, $\cdot$, $*$.

=> A.F. pt. $\{\forall\}$, $\emptyset$

Aplicând operațiile (cf. Th. închidere a L(A.F.)) => $\cup$, $*$, $\cdot$

ex: $L = (ab \cup aab)^*$



"⇐" L(A.F.) ⇒ L.R.

Idee: Vreau să arăt că L(M) este reuniunea unui nr. finit de L.R. (R)

Fie $M = (K, \Sigma, \delta, s, F)$

$K = \{q_1, \dots, q_m\}$, $s = q_1$

$R(i, j, k)$ e mult. șirurilor care duc automatul din starea q_i în starea q_j fără să treacă printr-o stare interm. $\geq k$

$R(i, j, k) = \{x \in \Sigma^* \mid (q_i, x) \overset{*}{\underset{M}{\vdash}} (q_j, y)$ unde $\ell < k$ sau $\ell = j$ și $y = e$ sau $\ell = i$ și $x = y\}$

$R(i, j, x+1) = \{x \in \Sigma^* \mid (q, x) \overset{*}{\underset{M}{\vdash}} (q_j, e)\}$, $q_j \in F$

$L = \bigcup R(1, j, x+1)$, $q_j \in F$

⇓

R regulat

Inductie : finit => regulat

Dem. prin inducție

$$R(i, j, 1) = \begin{cases} \{x \mid (\delta(q_i, x) = (q_j, e), i \neq j\} \\ \{e\} \cup \{x \in \Sigma \mid \delta(q_i, x) = (q_j, e), i = j\} \end{cases}$$

finit ⇒ regulat

Ip. ind. → adev. pt. k

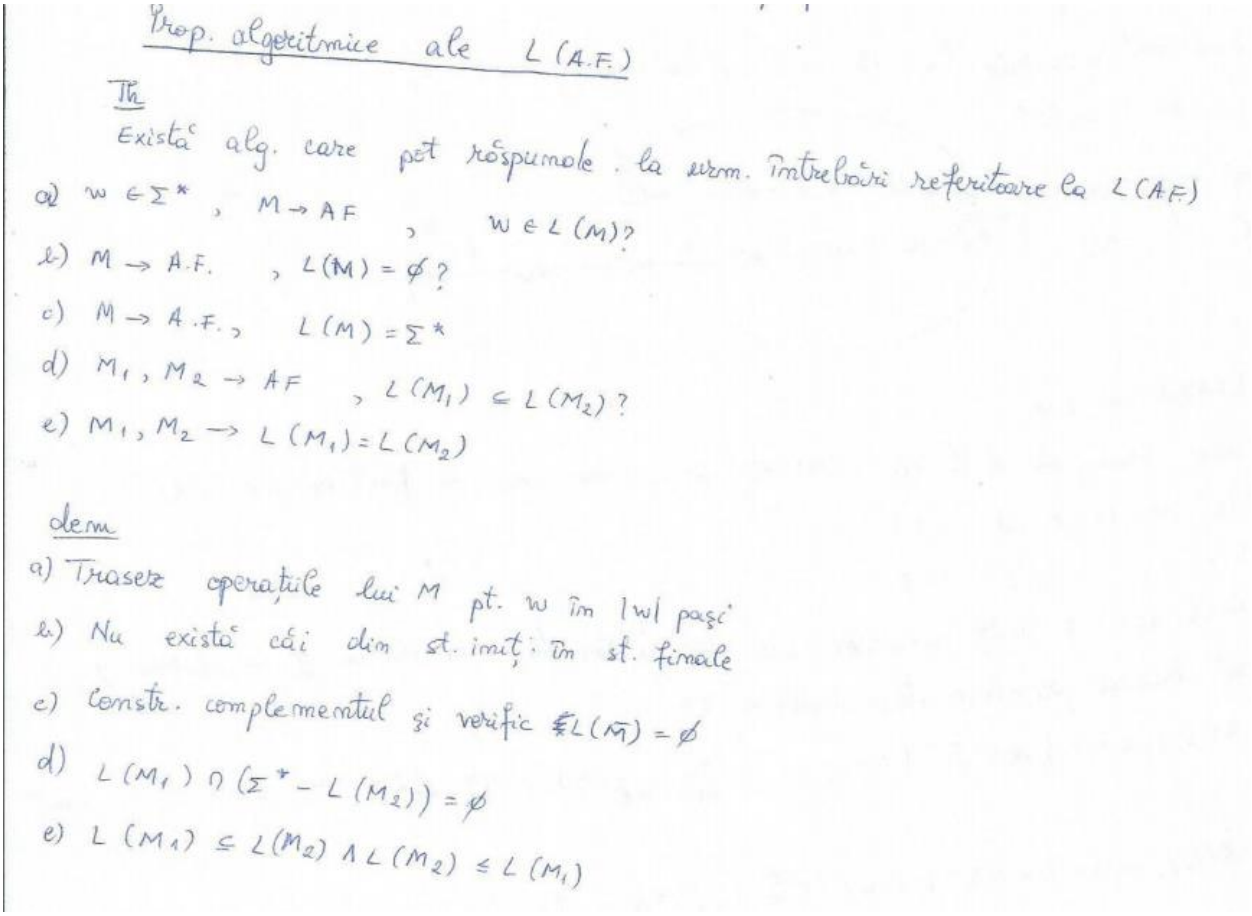
Pas de inducție → k+1

$R(i, j, k+1) = \underbrace{R(i, j, k)}_{\text{ip. ind.}} \cup \underbrace{R(i, k, k)}_{\text{ip ind.}} \underbrace{R(k, k, k)^*}_{\text{ip. ind.}} \underbrace{R(k, j, k)}_{\text{ip. ind.}}$ → pt. toate ține ipoteza inductivă ⇒

regulat (conf. prop. L.R)

Proprietati algoritmice ale L (AF)

Teorema : Exista algoritmi care pot raspunde la urmatoarele intreabr referitoare la L (AF)



Prop. algoritmice ale L (A.F.)

Th

Există alg. care pot răspunde la urm. întrebări referitoare la L (A.F.)

a) $w \in \Sigma^*$, $M \to AF$, $w \in L(M)$?

b) $M \to A.F.$, $L(M) = \emptyset$?

c) $M \to A.F.$, $L(M) = \Sigma^*$

d) $M_1, M_2 \to AF$, $L(M_1) \subseteq L(M_2)$?

e) $M_1, M_2 \to L(M_1) = L(M_2)$

dem

a) Trasez operațiile lui M pt. w în |w| pași

b) Nu există căi din st. inițiale în st. finale

c) Constr. complementul și verific $\in L(\overline{M}) = \emptyset$

d) $L(M_1) \cap (\Sigma^* - L(M_2)) = \emptyset$

e) $L(M_1) \subseteq L(M_2) \wedge L(M_2) \subseteq L(M_1)$

Tehnici de demonstrare a regularitatii limbajelor regulate :

Ex: Limbajul nr nat poz divizibil cu 2 sau 3 -> regulat

Tehnici de demonstrare a regularitatii limbajelor regulate

ex : Limbajul nr. nat. poz. divizibile cu 2 sau 3 → regulat

2, 6, 14 ∈ L

001, 5 ∉ L

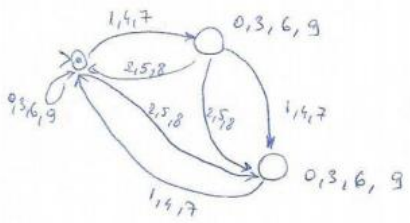
$\Sigma = \{0, 1, \dots, 9\}$

$L_1 = \{0\} \cup \{1, \dots, 9\}\ \Sigma^*$

$L_2 = L_1 \cap \Sigma^*$ $\{0, 2, 4, 6, 8\}$

$L_3 = L_1 \cap L(M)$

↓ nr. divizibile cu 3



ex $\Sigma = \{a, b\}$

$L = \{w \in \Sigma^* \mid |w|$ impară, w conține nr. par de $a\}$

$L_1 = \Sigma(\Sigma\Sigma)^*$

$L_2 = b^*(ab^*ab^*)^*$

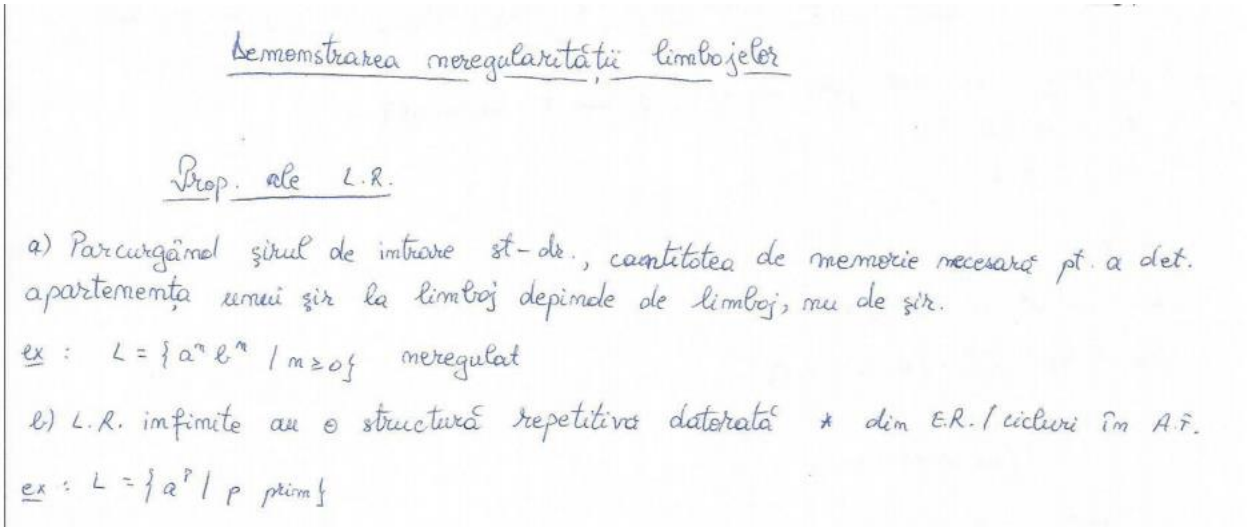
$L = L_1 \cap L_2$

Demonstrarea neregularitatii limbajelor :

Proprietati ale Limbajelor regulate :

-Parcurgand sirul st dr, cantitatea de memorie necesara pt a determia apartenenta unui sir la un limbaj depinde de limbaj, nu de sir.

-Limbajele regulate au o structura repetitiva datorata * din expresiile regulate / cicluri in automate finite



Demonstrarea neregularității limbajelor

Prop. ale L.R.

a) Parcurgând șirul de intrare st-dr., cantitatea de memorie necesară pt. a det. apartenența unui șir la limbaj depinde de limbaj, nu de șir.

ex: $L = \{a^n b^n \mid n \geq 0\}$ neregulat

b) L.R. infinite au o structură repetitivă datorată * din E.R. / cicluri în A.F.

ex: $L = \{a^p \mid p \text{ prim}\}$

Lema de pompare

Fie L un limaj regulat infinit. Atunci exista sirurile :

Th (Lema pompare)

Fie L un limbaj regulat infinit. Atunci $\exists$ şirurile $x, y \neq e, z$ a.î. $x y^m z \in L$, $\forall m \in \mathbb{N}$.

dem: L → limbaj regulat $\overset{\text{Th. echiv}}{\Rightarrow}$ $\exists M$, AFD a.î. $L = L(M)$

Pp. că M are n stări.

L → limbaj infinit $\Rightarrow$ $\exists w \in \Sigma^*$, $|w| = p \geq n$

Fie $w = \sigma_1 \sigma_2 \cdots \sigma_p$, $\sigma_1, \dots, \sigma_p \in \Sigma$

$w \in L$

$(q_0, \sigma_1 \dots \sigma_p) \vdash_M (q_1, \sigma_2 \dots \sigma_p) \vdash_M \cdots \vdash_M (q_p, e)$, $q_p \in F$

M are n stări $\Rightarrow$ $\exists\ 0 \leq i < j < n$ şi $\sigma_{i+1} \dots \sigma_j$ care duce M din q_i înapoi în q_i (Principiul lui Dirichlet / cuiburi)

$$\underbrace{\sigma_1 \sigma_2 \dots \sigma_i}_{x} \underbrace{(\sigma_{i+1} \dots \sigma_j)}_{y} \underbrace{\sigma_{j+1} \dots \sigma_p}_{z}$$

$$(q_0, x y^n z) \vdash_M^* (q_i, y^n z) \vdash_M^* (q_i, y^{n-1} z) \vdash_M^* \cdots \vdash_M^* (q_i, z) \vdash_M^* (q_p, e)$$

ex: $L = \{ a^n b^n \mid n \geq 0 \}$?

Pp. L → regulat, infinit $\Rightarrow$ aplic lema de pompare

$\exists\ x, y \neq e, z$ a.î. $x y^k z \in L$, $\forall k \geq 0$

I $\quad x = a^p$, $y = a^q$, $z = a^r b^s$ $\quad \Rightarrow x y^k z = a^p (a^q)^k a^r b^s = a^{p+qk+r} b^s$

Activați Windows

II $x = a^p$, $y = a^2 b^k$, $z = b^s$ $\Rightarrow xy^k z = a^p (a^2 b^k)^k b^s = a^p \underbrace{a^2 b^k a^2 b^k \dots a^2 b^k}_{k \text{ ori}} b^s \notin L$

III $x = a^p$, $y = b^k$, $z = b^s$ $\Rightarrow xy^k z = a^p b^2 (b^k)^k b^s = a^p b^{2+k \cdot k+s} \Rightarrow p = 2 + k \cdot k + s$, $\forall k \geq 0$ fals

Am ar. că nu-am cum să găsesc un pct. de pompare, iar lbj. să rămână regulat.

$\Rightarrow$ L - regulat $\rightarrow$ fals

Am aratat ca nu am cum sa gasesc un punct de pompare, iar limbajul sa ramana regulat.

Ex : Pp L limaj regulat infinit -> aplicam lema de pompare

ex: $L = \{a^p \mid p \text{ prim}\}$?

Pp $L \rightarrow$ regulat infinit

Aplic lema de pompare $\Rightarrow \exists x, y \neq e, z$ a.i. $x y^m z \in L$, $\forall m \geq 0$

$x = a^p$, $y = a^R$, $z = a^s$ $\Rightarrow a^{p + Rm + s} \Rightarrow a^p$, p prim, $\forall m \geq 0$, fals

O altă formă a lemei de pompare (LP):

Th

Fie $L \rightarrow$ lbj regulat, infinit, atunci $\exists$ n dependent de L și $x, y \neq e, z$ a.î. $|xy| \leq n$ și $xy^k z \in L$, $\forall k \geq 0$

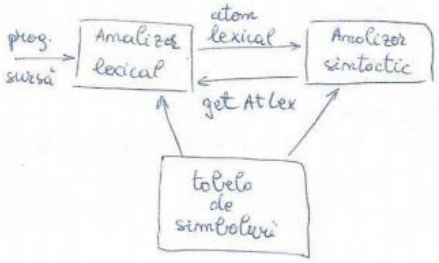
! EXAMEN : de demo. lema de pompare pt. lbj. regulate

ANALIZA LEXICALA :

Atom lexical / sablon / lexema

Limbaje pompare : atomi lexicali, cuvinte cheie, operator, constante, identificatori, siruri literali, simboluri de punctuatie

Analiza lexicală



Atom lexical, şablon, lexemă

Atom lexical → categoria asoc. unei mulţimi de şiruri

Lexemă → şir

Şablon → regula prin care verific clasificarea unui şir

ex: const pi =3.14;

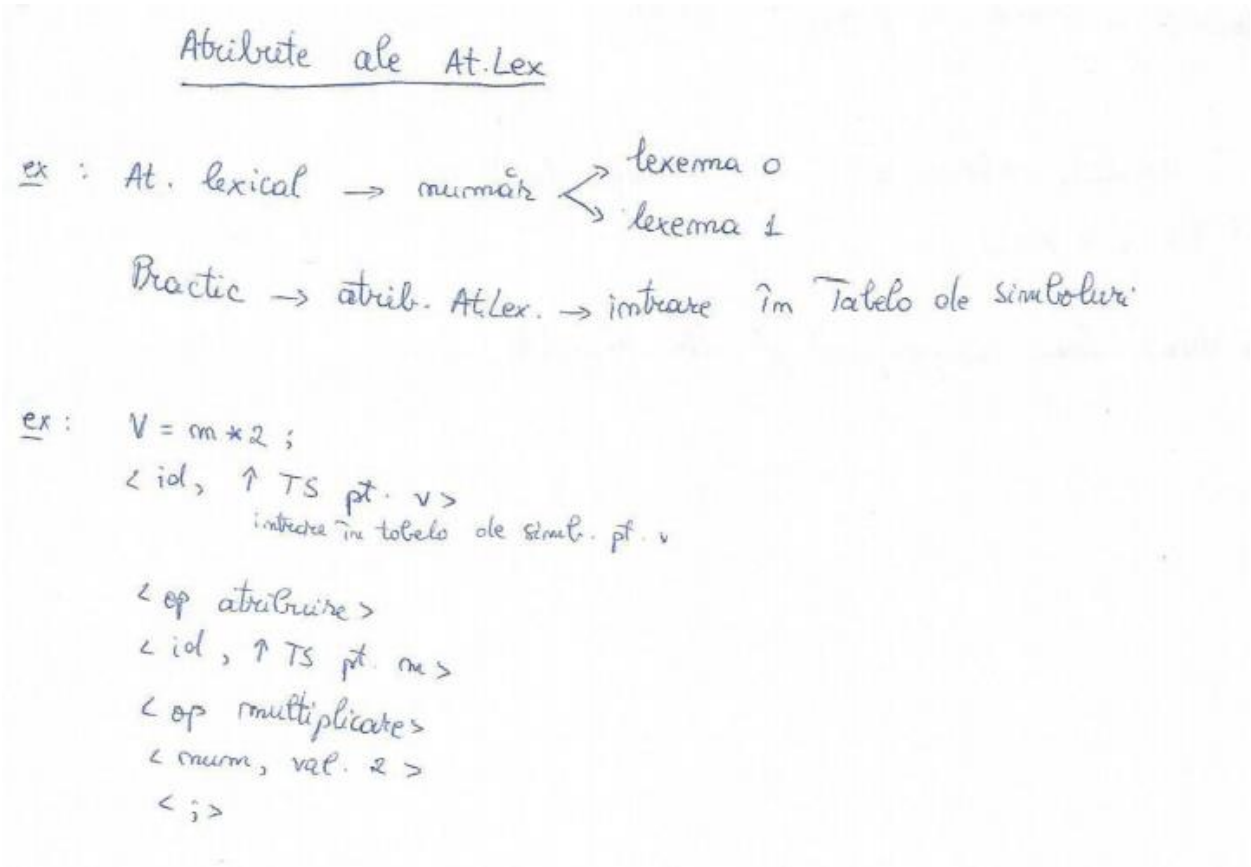
pi → lexema pt. atom lexical "identific"

Limbaje programare → at. lexicali, cuv. cheie, operatori, constante, identificatori, şiruri literali, simboluri de punctuaţie etc.

Activaţi Windows

ATRIBUTE ALE ATOMULUI LEXICAL :

Tabela de simboluri / lexema



Definitii Regulate :

Identificatorii sunt siruri de litere sau cifre care incep cu o litera

Constantele numerice fara semn

Definiții Regulate

Șablon → Expr. Reg.

$d_1 \to r_1$
$d_2 \to r_2$
.....
$d_m \to r_m$

peste Σ, r_i este def. peste $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$

ex: identificatorii sunt șiruri de litere sau cifre care încep cu o literă

lit → A | ... | Z | a | ... | z

cifră → 0 | 1 | ... | 9

id → lit (lit | cif)*

ex: const. numerice fără semn: 2350, 4.99, 5.07 E-3

cif → 0 | 1 | ... | 9 cif1 → 1 | ... | 9

cifre → cif1 (cif)*

partea fracționară → . cifre | e

partea exponențială → E (+ | - | e) cifre | e

num. → cifre p-fract p-exp

Notatii :

Notații

$r+ \equiv rr^*$

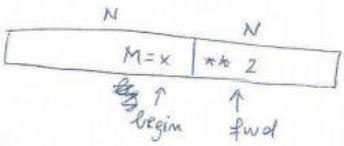
$r? \equiv r|e$

$[abc] \equiv a|b|c$

$[a-z] \equiv a|b|...|z$

Opțiuni implementare - bufferare intrare

buffer



N (1024, 4096)

Inițial → beg ≡ fwd

Deplasez fwd până la identific. unei lexeme ; fwd - primo poz. după lexemei

După prelucrarea lexemei : beg ← fwd

buffer → circular

ANALIZA STATICA :

- Delimitator
- Identificator
- Num
- Analizator lexical -> atom lexical + atribute

instr → if expr. there instr else instr

| if expr then instr

| e

op. rel - operator relațional

expr → term ~~op. rel~~ term

| term

analizor
~~analizator~~
sintactic

term → num | id

if → if

then → then

else → else

oprel → < | <= | > | >= | = | <>

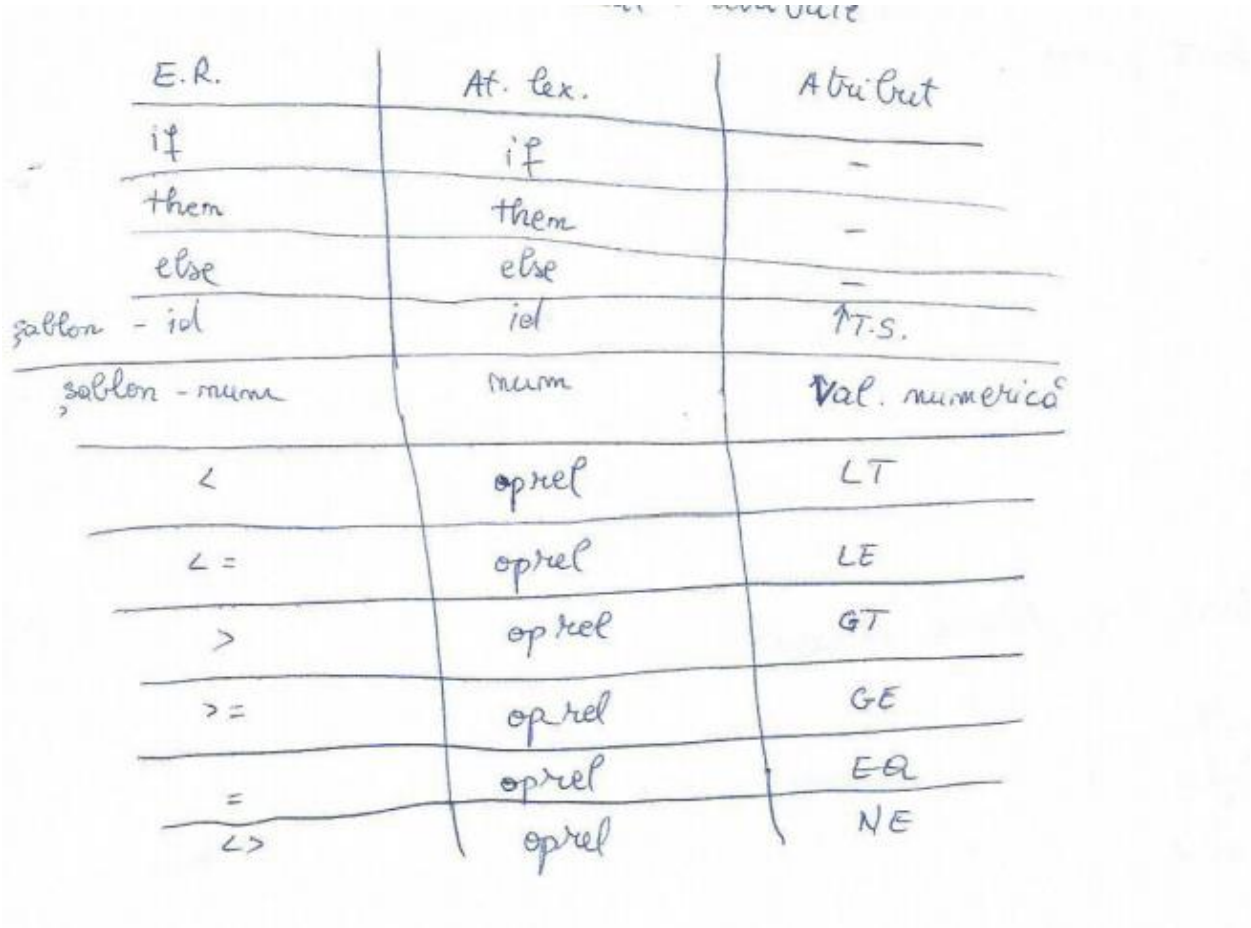
id → lit (lit | cif)*

num → cif | cif* (. cifre)? (E (+/-)? cifre)?

delim → blank | tab | new line

ws → delim+

Am. lexical → at. lexical + atribute



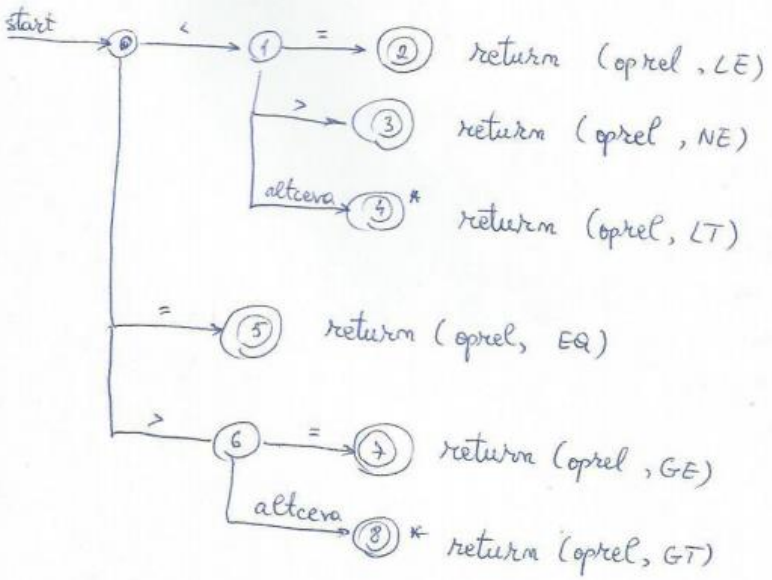
E.R.	At. lex.	Atribut
if	if	-
them	them	-
else	else	-
şablon - id	id	↑T.S.
şablon - numa	numa	Val. numerică
<	oprel	LT
< =	oprel	LE
>	oprel	GT
> =	oprel	GE
=	oprel	EQ
< >	oprel	NE

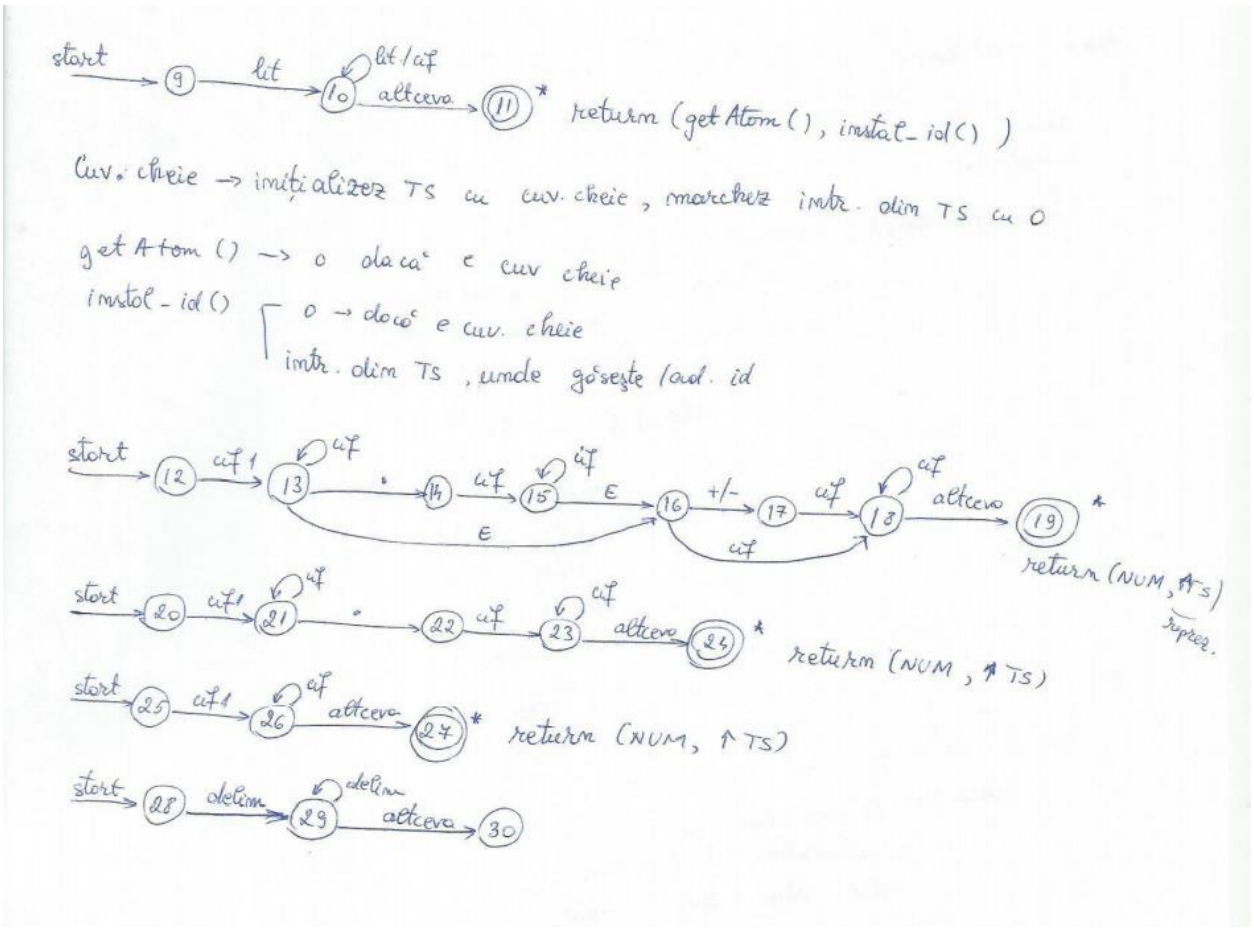
Definirea AF : Diagrame de tranzitii :

La recunoasterea unui Atom Lexical -> atom + atribut

Definirea AF → diagrame de tranziții

La recunoașterea unui AtLex → atom + atribut





IMPLEMENTARE :

Implementare

Am : nextchar() → citește câte un caracter din buffer, avans. ~~forward~~ fwd

```
int stare = 0, start = 0;
int fail()
{
   fwd = beg ;
   switch (start) {
      case 0 :  start = 9 ; break;
      case 9 :  start = 12 ; break;
      case 12 : start =20; break;
      case 20 : start = 25 ; break;
      case 25 : recover (); break;
      default;  // ERR comp
   } // end switch
   return start;
}
```

```
Aterm nextAterm ()
{
    char c;
    while (1)
    {
      switch (state) { case 0: c = next char();
                       if (c==' ' || c=='\t' || c=='\n'){
                           c = next char();
                           beg++;
                       }
                       else if (c == 'c')
                             state = 1;
                             break;
                          else if ( c== '='
                                state = 5;
                             else if (c=='s')
                                state 6;
                                else
                                 state = fail();
                                 break;
      case 9 : c= next char ();
               if (is letter (c)) state =10;
               else state = fail (); break;
```

```
case 10 :   c = nextchar ();
            if (isletter (c) || isdigit (c))   state = 10;
            else state = 11; break;

case 11 :  retract (); instal_id () ; return getAtom ();

. . .

case 25 :  c = nextchar ();
            if (isdigit (c) && (c - '0' > 0))] state = 26 ; break;
            else state = fail (); break;

case 26 :  c = nextchar ();
            if (isdigit (c))   state = 26 ;
            else state = 27; break;
case 27 :  retract () ; return NUM;
        }
    }
}
```